Università degli Studi di Napoli Federico II

Corso di Ingegneria del Software



BugBoard26

Documentazione di progetto

Requisiti software, design del sistema, sviluppo, testing e usabilità

Gruppo INGSW2526_048

Matricole N86004715, N86004797

Nomi degli autori: da completare

21 ottobre 2026

Indice

Parte I Documento dei Requisiti Software....4
1 Glossario....4
2 Introduzione....5
3 Requirement Elicitation and Analysis....6
3.1 Individuazione del target degli utenti tramite Personas....6
4 Requirement Specification....9
4.1 Use Case Diagram....9
4.2 Requisiti Funzionali....10
4.3 Requisiti Non Funzionali....12
4.4 Requisiti di Dominio....13
4.5 Rappresentazione Tabellare Modello di A.Cockburn....14
Parte II Documento di Design del Sistema....21
5 Progettazione Architetturale e Scelte Tecnologiche....21
5.1 Descrizione dell'Architettura....21
5.1.1 Controller....21
5.1.2 Service....21
5.1.3 Model e accesso ai dati....21
5.2 Architettura del Frontend....22
5.3 Criteri di Design e Motivazioni....23
5.4 Tecnologie Utilizzate....24
5.5 Schema della Persistenza dei Dati....25
5.5.1 Descrizione delle entità e delle relazioni....26
6 Design dell'interfaccia utente....27
6.1 Progettazione Cromatica dell'Interfaccia....27
6.1.1 Obiettivi della Progettazione Cromatica....27
6.1.2 Palette Cromatica Adottata....27
6.1.3 Applicazione della Regola 60–30–10....28
6.1.4 Accessibilità e Contrasto Cromatico....28
6.2 Tipografia....29
6.2.1 Obiettivi della Progettazione Tipografica....29
6.2.2 Scelta del Font....29
6.2.3 Gerarchia Tipografica....29
6.2.4 Spaziatura e Allineamento....29
6.2.5 Accessibilità e Conformità alle WCAG....29
6.2.6 Coerenza con l'Identità Visiva....29
6.3 Principles of Proximity and Connectedness....30
6.4 Applicazione del modello F-Pattern....33
6.5 Labor Perception Bias....34
6.6 Gestione del Feedback Utente e Feedback Persistence....34
6.7 Prevenire gli Errori....36
7 Diagrammi delle classi di Design....37
7.1 Classi di Design del Backend....37

7.1.1 Architettura Controller–Service–Model....37
7.1.2 DTO....38
7.1.3 Entità di Dominio e sicurezza....42
7.2 Classi di Design del Frontend....43
7.2.1 Utility, Config e Security....49
7.3 Gestione della navigazione tra le pagine....51
7.4 Sequenze di Design per gli Use Case....54
7.4.1 UC1 – Creazione issue....54
7.4.2 UC2 – Cambio stato....54
7.4.3 UC3 – Commento....55
7.4.4 UC4 – Creazione account e appartenenza....55
Parte III Documentazione del Processo di Sviluppo....56
8 File di build automatica....56
8.1 DockerFile (Backend)....56
8.2 DockerFile (Frontend)....56
8.3 Docker Compose....57
9 Strumento di Versioning....58
10 Qualità del codice....59
10.1 Grafico di Duplicazione del Codice....59
10.2 Grafico delle Issues....59
10.3 Grafico della Coverage dei Test....59
Parte IV Testing e Valutazione dell'Usabilità....60
11 Unit Testing....60
11.1 Test di createIssue (issues service)....60
11.2 Test di changeStatus (issues service)....61
11.3 Test di requireAssignee (access service)....62
11.4 Test di notify (notifications service)....63
11.5 Strategie adottate....64
11.5.1 Controller....64
11.5.2 Service....64
11.5.3 Model / persistenza....64
11.5.4 Security e Utils....64
11.6 Test plan di una funzionalità: risoluzione issue....65
12 Valutazione dell'Usabilità....66
12.1 Expert Review....66
12.1.1 Obiettivo del Test....66
12.1.2 Istruzioni per l'Uso della Checklist....66
12.1.3 Risultati....66
12.2 Relazione Sperimentale....68
12.2.1 Descrizione dei Partecipanti....68
12.2.2 Procedura Sperimentale....68
12.2.3 Dati Raccolti....69
12.2.4 Valutazione post-task....69
12.2.5 Conclusioni....70

Parte I Documento dei Requisiti Software

1 Glossario

Termine	Significato
Utente	Account autenticato che opera nei progetti accessibili; ruolo globale USER o ADMIN.
Amministratore	Account ADMIN che crea utenti, progetti e appartenenze e ha accesso globale ai progetti. Il cambio stato richiede comunque di essere assegnatario.
Progetto e appartenenza	Un progetto raggruppa issue e membri. Membership collega un account a un progetto e limita l'accesso degli USER.
Issue	Segnalazione con titolo, descrizione e tipo. BUG, QUESTION, DOCUMENTATION e FEATURE distinguono malfunzionamenti, domande, documentazione e nuove funzionalità.
Reporter	Utente che crea la issue; viene ricavato dall'identità autenticata e non scelto dal client.
Assegnatario	Membro incaricato di una issue. L'assegnazione è facoltativa; il solo assegnatario corrente può avanzare lo stato.
Priorità	Valore facoltativo LOW, MEDIUM, HIGH o CRITICAL; quando assente è NULL e la UI mostra Not specified.
Stato	Avanzamento TODO, IN_PROGRESS, CLOSED. Non è prevista la riapertura.
Commento	Messaggio di discussione con autore e data stabiliti dal server.
Notifica	Messaggio personale persistente di attività o risoluzione, con stato letto/non letto.
Allegato	Un'immagine facoltativa PNG, JPEG o WebP associata alla issue e scaricabile tramite API protetta.
API e DTO	Le API HTTP scambiano oggetti JSON. I DTO sono i contratti di questi oggetti; il backend li prepara con funzioni, senza classi DTO Java.
ORM e modello	Sequelize collega le classi Model alle tabelle PostgreSQL. Definire un modello non salva dati: create, update e destroy eseguono le scritture.
JWT e sessione	Il JWT è il token firmato inviato come Bearer. La sessione del browser conserva token e utente; non è una sessione server Express.
Transazione	Più scritture sono confermate insieme oppure annullate mediante rollback.

2 Introduzione

BugBoard26 è una piattaforma web per la gestione collaborativa delle issue nei progetti software. Il sistema permette di segnalare problemi e richieste, consultarne l'avanzamento, assegnarli ai membri del team e discutere le attività di risoluzione attraverso commenti.

L'attività universitaria comprende analisi e specifica dei requisiti, progettazione, implementazione e parziale verifica del sistema. Questa relazione raccoglie le quattro parti: requisiti software, design del sistema, processo di sviluppo e testing con valutazione dell'usabilità.

Le funzionalità assegnate dalla traccia sono i punti 1, 2, 3, 5 e 6:

- Autenticazione con email e password; account amministratore iniziale e creazione di account USER o ADMIN.
- Segnalazione di issue con titolo e descrizione; tipi QUESTION, BUG, DOCUMENTATION e FEATURE; priorità facoltativa e possibilità di allegare un'immagine; stato iniziale TODO.
- Vista riepilogativa con criteri di filtro o ordinamento utili alla consultazione.
- Commenti associati alle segnalazioni per messaggi, aggiornamenti e chiarimenti.
- Aggiornamento dello stato da parte dell'assegnatario e notifica al reporter quando la issue è risolta.

Il contesto di progetto già implementato è mantenuto: gli USER possono segnalare e commentare nei progetti di appartenenza; gli ADMIN hanno accesso globale. Il ruolo ADMIN non permette di cambiare lo stato di una issue assegnata ad altri. Queste regole valgono nel backend anche per richieste inviate fuori dall'interfaccia.

L'implementazione corrente usa React/Vite, Node.js/Express, Sequelize e PostgreSQL. Sono comprese le immagini e i controlli sul solo assegnatario. Ricerca globale, profilo modificabile, recupero password, abbandono ed eliminazione dei progetti e storico delle issue non fanno parte delle funzionalità implementate. Alcuni compaiono nei mockup progettuali e sono distinti dalle schermate reali.

3 Requirement Elicitation and Analysis

3.1 Individuazione del target degli utenti tramite Personas

Gli utenti target sono membri dei team di sviluppo, persone che segnalano problemi e amministratori che organizzano account e progetti. Le personas aiutano a descriverne obiettivi e bisogni. Le schede seguenti rappresentano profili di progetto, non partecipanti a una valutazione sperimentale già svolta.

Giulia Bianchi

Sviluppatrice: consulta le issue assegnate, ne comprende il contesto e aggiorna il lavoro svolto.



Figura 3.1 Persona di riferimento: Giulia Bianchi

Marco Ferri

Amministratore: crea account, organizza i progetti e mantiene le appartenenze.



Figura 3.2 Persona di riferimento: Marco Ferri

Luca Lauro

Utente e segnalatore: descrive il problema, allega una schermata e segue la risposta del team.



Figura 3.3 Persona di riferimento: Luca Lauro

4 Requirement Specification

4.1 Use Case Diagram

Il diagramma riporta utente, amministratore e assegnatario, insieme alle funzioni di segnalazione, consultazione, commento e gestione dei progetti. La specializzazione Assegnatario dipende dalla singola issue e non costituisce un terzo ruolo globale.

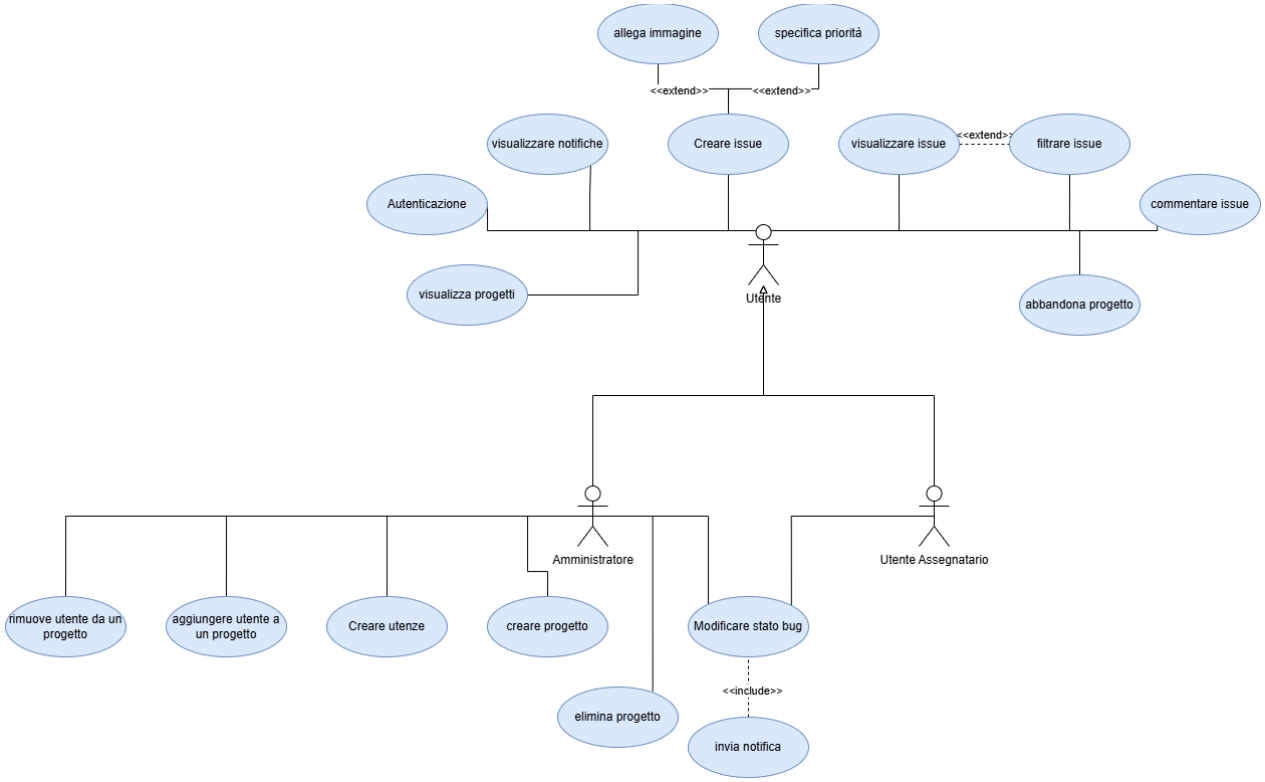


Figura 4.1 Diagramma dei casi d'uso del progetto

Lettura rispetto alle regole implementate: il collegamento dell'amministratore a Modificare stato bug è valido soltanto quando l'amministratore è anche assegnatario. Abbandona progetto ed elimina progetto restano casi della proposta iniziale privi di endpoint. L'immagine facoltativa è invece implementata. La tabella seguente definisce il comportamento effettivo.

4.2 Requisiti Funzionali

ID	Denominazione	Descrizione e stato
RF1	Login Utente	Accesso con email e password valide.
RF2	Logout Utente	Eliminazione della sessione locale e del Bearer dal client; il JWT non viene revocato immediatamente sul server.
RF3	Validazione Credenziali	Credenziali errate producono un errore uniforme, senza distinguere email assente e password errata.
RF4	Creazione Utente (Admin)	ADMIN crea un account con nome completo, email unica, password e ruolo USER o ADMIN.
RF5	Gestione Ruolo Utente	Ruoli globali riletti dal database e funzioni amministrative protette.
RF6	Creazione Issue	Un membro del progetto o un ADMIN può segnalare in un progetto accessibile.
RF7	Titolo e Descrizione	Entrambi obbligatori e validati anche dal backend.
RF8	Classificazione Issue	Tipo obbligatorio BUG, FEATURE, DOCUMENTATION o QUESTION.
RF9	Stato Iniziale	Il server imposta TODO e ricava reporter e date senza accettarli liberamente dal client.
RF10	Priorità	Facoltativa e nullable; valori LOW, MEDIUM, HIGH, CRITICAL.
RF11	Allegato Immagine	Una PNG, JPEG o WebP facoltativa entro 5 MB e 16 megapixel; controllo dei byte e download autenticato.
RF12	Elenco Issue	Consultazione delle issue del progetto accessibile.
RF13	Dettaglio Issue	Dati della issue, immagine, commenti e informazioni di assegnazione.
RF14	Filtraggio Issue	Filtri combinabili nel browser per tipo, stato e priorità, incluse quelle non specificate.
RF15	Ordinamento Issue	Ordine API per updatedAt e id decrescenti; nessun criterio selezionabile nella UI.
RF16	Visualizzazione Commenti	Commenti della issue in ordine cronologico crescente.
RF17	Inserimento Commento	Utente autorizzato al progetto; autore e data stabiliti dal server.
RF18	Aggiornamento Stato	Solo l’assegnatario autorizzato può avanzare TODO a IN_PROGRESS e poi CLOSED.
RF19	Controllo Modifica	403 per ogni non assegnatario, incluso ADMIN; richieste ripetute dello stesso assegnatario idempotenti.
RF20	Notifica Risoluzione	CLOSED genera RESOLUTION al reporter anche se coincide con l’attore o è ex membro.
RF21	Notifiche Personali	Lettura e marcatura soltanto delle proprie notifiche; issueId mascherato quando l’accesso è revocato.

ID	Denominazione	Descrizione e stato
RF22	Visualizzazione Progetti	USER vede i propri progetti; ADMIN ha accesso globale.
RF23	Selezione Progetto	Apertura della dashboard del progetto selezionato.
RF24	Aggiunta Membro	ADMIN sceglie un account esistente; l'API riceve userId e non cambia il ruolo globale.
RF25	Abbandono Progetto	Presente nella proposta iniziale; non implementato e fuori dall'ambito concordato.
RF26	Rimozione Membro	ADMIN elimina la membership; l'accesso degli USER viene revocato alle richieste successive.
RF27	Creazione Progetto	ADMIN indica nome e descrizione; progetto e membership del creatore sono salvati insieme.
RF28	Eliminazione Progetto	Presente nella proposta iniziale; non implementata e fuori dall'ambito concordato.

RF25 e RF28 mantengono gli identificativi della proposta per renderne tracciabile il perimetro; non rappresentano funzioni disponibili o nuovi obblighi di implementazione. RF1-RF5 ricadono nel punto 1 della traccia; RF6-RF11 nel punto 2; RF12-RF15 nel punto 3; RF16-RF17 nel punto 5; RF18-RF21 nel punto 6. RF22-RF28 descrivono il contesto dei progetti.

4.3 Requisiti Non Funzionali

Denominazione	Requisito e riscontro
Unicità Email	Una sola utenza per email normalizzata; controlli applicativi e vincolo SQL.
Unicità Appartenenza	Una coppia projectId/userId non può apparire due volte in project_memberships.
Sicurezza Credenziali	Hash BCrypt; JWT HS256 verificato per firma, scadenza, issuer e audience. Nessun hash nelle risposte pubbliche.
Controllo Accessi	Autenticazione, ruolo, appartenenza al progetto e identità dell'assegnatario verificati lato server.
Integrità dei Dati	Validazione dei campi, vincoli PostgreSQL e transazioni delle scritture composte; test di rollback.
Persistenza	Dati mantenuti dopo riavvio del processo; immagini salvate come BYTEA nella stessa base dati.
Prestazioni e Tempo API	Lettura asincrona e immagini separate dalle liste; tempi sotto carico non ancora misurati.
Scalabilità	Frontend e backend separati. Liste non paginate e polling ogni 30 secondi sono limiti da valutare; nessuna capacità certificata.
Usabilità e Accessibilità	Feedback, raggruppamenti visivi e form guidati; review e test con utenti da completare.
Manutenibilità	Moduli con responsabilità riconoscibili, import/export, API centralizzate e test riproducibili.
Portabilità	Node.js, lockfile e Dockerfile backend; frontend compilato in asset statici con Vite.

4.4 Requisiti di Dominio

Vincolo	Regola applicata
Tipi e dati minimi	Quattro tipi previsti; titolo, descrizione, tipo e progetto obbligatori. Priorità, assegnazione e immagine facoltative.
Relazione Progetto-Issue	Ogni issue appartiene a un solo progetto e ha un solo reporter; un progetto può contenere più issue.
Assegnazione	Il destinatario deve essere membro corrente, anche se ADMIN. Qualsiasi utente con accesso alla issue può assegnare, riassegnare o rimuovere l'assegnazione.
Modifica dello Stato	Solo assegnatario corrente con accesso; TODO a IN_PROGRESS a CLOSED. Nessun salto, ritorno o riapertura.
Idempotenza	Stesso stato richiesto dall'assegnatario non ripete aggiornamento o notifiche. Il controllo dell'identità precede l'idempotenza.
Revoca Accesso	Un USER rimosso dal progetto perde accesso a issue, immagini e commenti. I riferimenti storici a reporter o assegnatario restano.
Commenti	Messaggi anche dopo la chiusura; autore server e limite di 5000 caratteri.
Notifica Risoluzione	Sempre al reporter, anche se attore o ex membro. In caso di revoca il messaggio non concede accesso al dettaglio.
Atomicità	Issue, immagine, attività di progetto e notifiche sono confermate nella stessa transazione. Commento e notifiche seguono la stessa regola.
Dati Centralizzati	Il backend è la fonte dei dati persistenti. Il browser mantiene soltanto sessione e stato di presentazione.

4.5 Rappresentazione Tabellare Modello di A.Cockburn

Le quattro schede seguono gli identificativi UC1-UC4 del progetto. I mockup descrivono il design dell'interazione; le schermate reali permettono di verificarne la realizzazione. Il codice applica commenti e avanzamento a tutti i tipi di issue, oltre al caso bug esplicitato dalla traccia.

UC1 - Creare Issue

Campo	Descrizione
Use Case	UC1 - Creare Issue
Goal in Context	Segnalare un problema o una richiesta con immagine facoltativa.
Preconditions	Sessione valida e progetto accessibile.
Primary Actor	Utente membro del progetto o ADMIN
Trigger	Selezione di Create Issue.
Success End Conditions	Issue persistente in TODO con reporter autenticato, eventuale immagine e notifica coerenti.
Failed End Conditions	Nessuna issue, immagine o notifica parziale.

Step	User Action	System
1	Apre il modulo.	Carica i membri del progetto.
2	Compila titolo, descrizione e tipo.	Consente priorità, assegnatario e file facoltativi.
3	Sceglie eventualmente un'immagine.	Mostra file e controlli iniziali; consente Remove image.
4	Conferma.	Valida dati, accesso, assegnatario e contenuto reale del file.
5	Legge il risultato.	Salva in transazione, risponde 201 e apre il dettaglio.

Extensions. Campi non validi o file falso/corrotto: 400. File oltre 5 MB: 413. Accesso revocato: 403. Scrittura o notifica fallita: rollback; dati nel form conservati. Cancel chiude senza salvataggio.

Subvariations e Notes. Senza immagine il client invia JSON; con immagine usa multipart. Priorità e assegnatario omessi restano null.

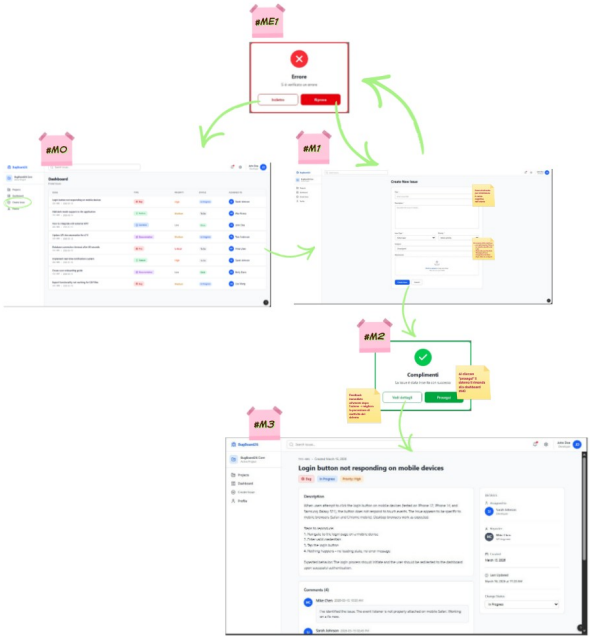


Figura 4.2 Mockup progettuale del flusso UC1; le regole applicate sono descritte nella scheda

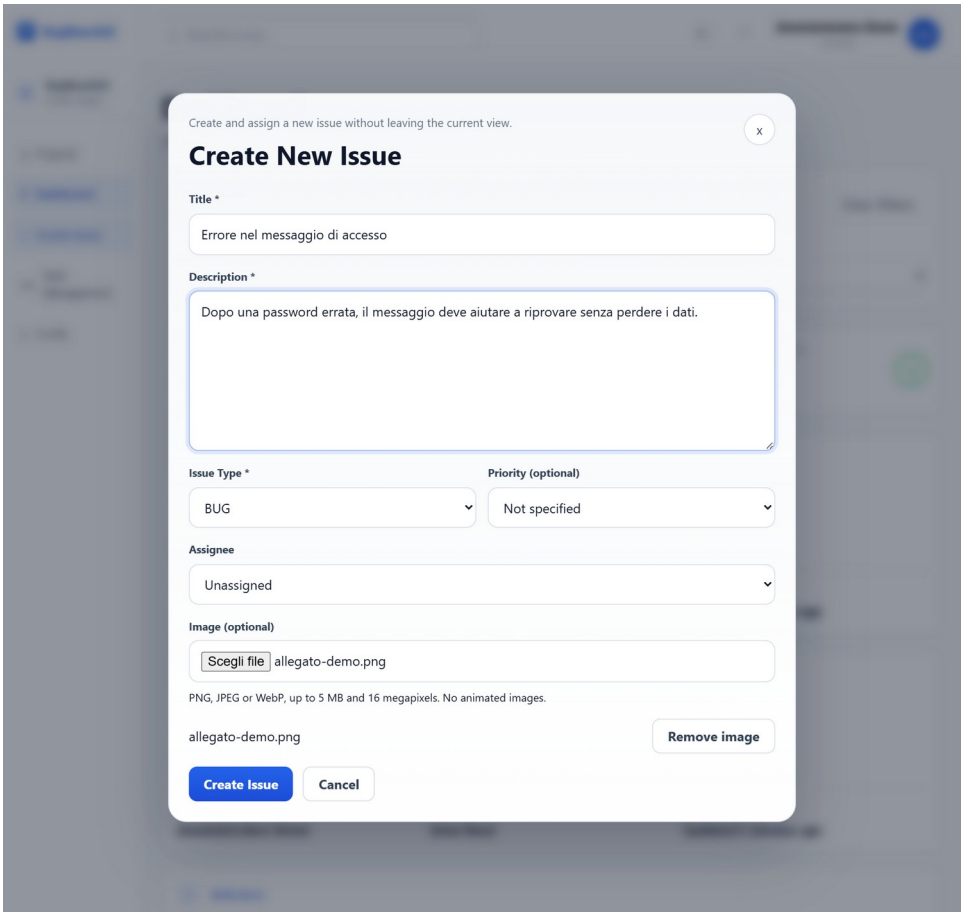


Figura 4.3 Realizzazione di UC1: creazione con immagine facoltativa

UC2 - Modificare stato bug

Campo	Descrizione
Use Case	UC2 - Modificare stato bug
Goal in Context	Registrare presa in carico e risoluzione della issue assegnata.
Preconditions	Sessione valida, issue accessibile e assegnata all’attore.
Primary Actor	Assegnatario corrente
Trigger	Selezione dello stato successivo.
Success End Conditions	Stato avanzato e, in chiusura, notifica RESOLUTION al reporter.
Failed End Conditions	Stato e notifiche restano invariati.

Step	User Action	System
1	Apre una issue assegnata.	Mostra stato e transizione disponibile.
2	Seleziona IN_PROGRESS da TODO.	Blocca la riga nella transazione e verifica identità e transizione.
3	Dopo la risoluzione seleziona CLOSED.	Aggiorna stato/progetto e notifica il reporter.
4	Legge la conferma.	Restituisce il dettaglio dopo commit.

Extensions. Non assegnatario, anche ADMIN: 403. Senza assegnatario: 403. Salto o ritorno di stato: 409. Fallimento notifica: rollback.

Subvariations e Notes. Ripetere lo stato corrente da assegnatario è idempotente. Non è prevista riapertura; la riassegnazione trasferisce subito la facoltà di avanzare.

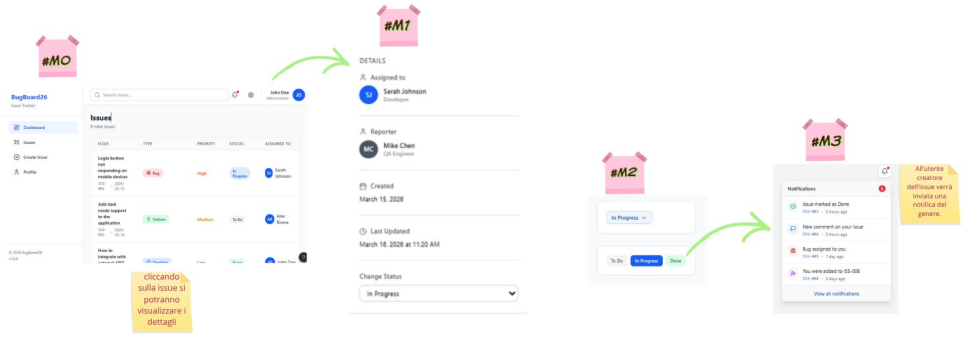


Figura 4.4 Mockup progettuale del flusso UC2; le regole applicate sono descritte nella scheda

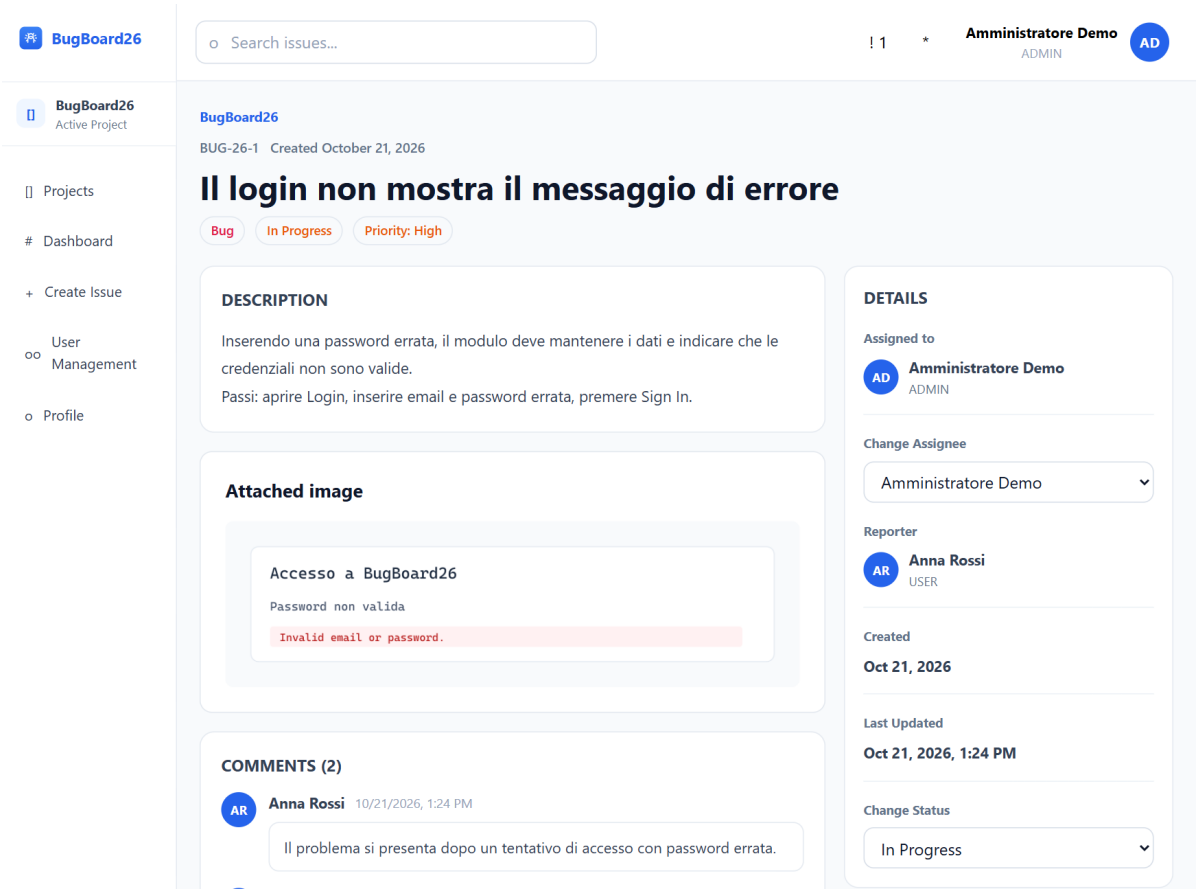


Figura 4.5 Realizzazione di UC2: dettaglio e controllo dello stato

UC3 - Inserire un commento

Campo	Descrizione
Use Case	UC3 - Inserire un commento
Goal in Context	Condividere messaggi, aggiornamenti o richieste di chiarimento.
Preconditions	Sessione valida e issue accessibile, anche chiusa.
Primary Actor	Utente membro del progetto o ADMIN
Trigger	Invio di Add Comment.
Success End Conditions	Commento con autore e data server; notifiche e attività aggiornate insieme.
Failed End Conditions	Nessun commento parziale; testo conservato nel form in caso di errore.

Step	User Action	System
1	Apre il dettaglio.	Carica la discussione in ordine cronologico.
2	Scrive e invia il messaggio.	Valida testo e accesso, salva con autore autenticato.
3	Consulta l’aggiornamento.	Aggiunge il commento e svuota il campo dopo il successo.

Extensions. Testo vuoto o oltre 5000 caratteri: 400. Esterno o membro rimosso: 403. Errore nelle notifiche: rollback.

Subvariations e Notes. Reporter e assegnatario ricevono le notifiche ordinarie solo se autorizzati, senza duplicati ed escludendo l’attore.

UC4 - Creare utenze

Campo	Descrizione
Use Case	UC4 - Creare utenze
Goal in Context	Fornire un account USER o ADMIN e, quando richiesto, aggiungerlo a un progetto.
Preconditions	Sessione ADMIN e pagina User Management aperta.
Primary Actor	Amministratore
Trigger	Invio del form Create New User.
Success End Conditions	Account persistente con password BCrypt; appartenenza aggiunta con una seconda richiesta.
Failed End Conditions	Una creazione account fallita non salva l'utenza; un'aggiunta membro fallita conserva l'account già creato.

Step	User Action	System
1	Indica nome, email, password e ruolo.	Valida i dati e verifica l'unicità dell'email.
2	Conferma la creazione.	Calcola l'hash e salva; risponde 201 con utente pubblico.
3	Aggiunge il nuovo account al progetto.	Esegue una richiesta separata di membership.
4	Legge l'esito.	Aggiorna membri e messaggi di feedback.

Extensions. Non ADMIN: 403. Email già usata: 409. Dati non validi: 400. Membership duplicata: 409. Se il secondo passo fallisce, usare Add Existing User.

Subvariations e Notes. Il ruolo è globale; aggiungere un account a un progetto non ne cambia i privilegi globali.

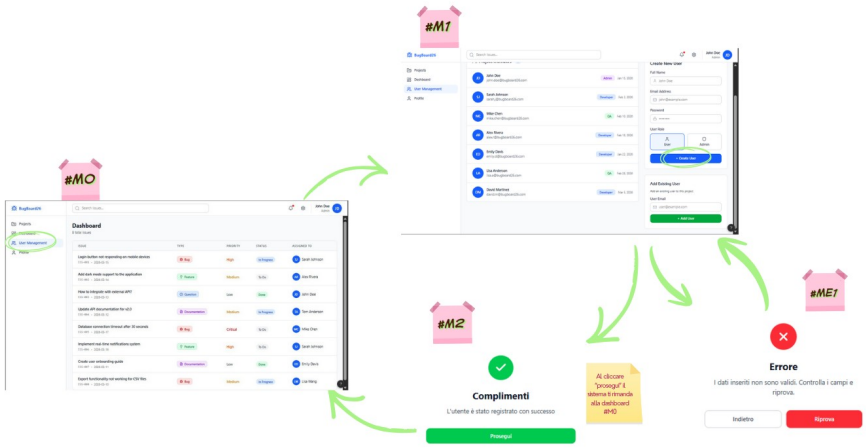


Figura 4.6 Mockup progettuale del flusso UC4; le regole applicate sono descritte nella scheda

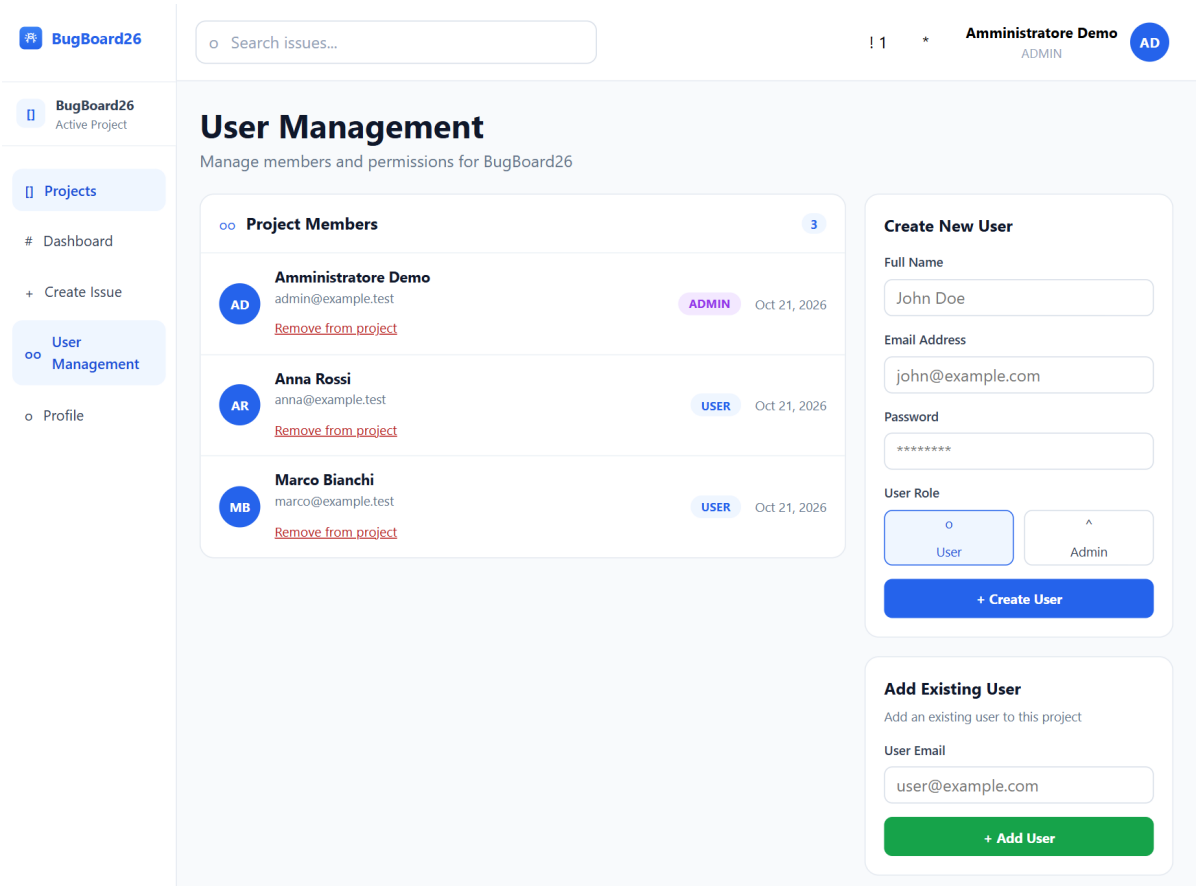


Figura 4.7 Realizzazione di UC4: gestione account e appartenenze

Parte II Documento di Design del Sistema

5 Progettazione Architetturale e Scelte Tecnologiche

5.1 Descrizione dell'Architettura

La soluzione separa il browser React, le API Node.js/Express e il database PostgreSQL. Il browser comunica tramite HTTP; soltanto il backend legge e modifica i dati persistenti. Nel backend il percorso è richiesta, rotta, middleware, controller, eventuali servizi, modelli Sequelize e risposta.

5.1.1 Controller

I controller ricevono body e parametri, selezionano i campi ammessi e preparano status e risposta HTTP. Il middleware rende disponibile req.user dopo il controllo JWT. Letture semplici usano direttamente i modelli; le operazioni composte sulle issue passano a services/issues.js. Il controller dei commenti coordina direttamente la transazione e i servizi condivisi.

5.1.2 Service

I servizi riuniscono regole riutilizzabili e operazioni composte. access.js controlla progetti, issue e assegnatari; auth.js gestisce password e token; issues.js esegue creazione, cambio stato e assegnazione; notifications.js seleziona i destinatari. images.js decodifica e normalizza il file, mentre responses.js prepara soltanto gli oggetti pubblici.

5.1.3 Model e accesso ai dati

Le sette classi Sequelize estendono Model e sono definite in models/index.js. Non esiste un livello repository separato. init descrive campi e mapping; findByPk e findAll eseguono letture, create/update/destroy eseguono scritture. Le operazioni legate da atomicità condividono la stessa transaction PostgreSQL.

5.2 Architettura del Frontend

Il frontend è una SPA React con componenti funzione. main.jsx inizializza l'applicazione; App e AuthProvider collegano contesto e router. AppRouter associa gli URL alle pagine, mentre RouteGuard impedisce la navigazione alle aree protette senza una sessione valida.

Le pagine compongono i componenti condivisi e mantengono lo stato locale mediante hook: filtri, input, caricamento e messaggi. AuthContext rende condivisi sessione e notifiche tramite useAuth. Gli adapter nella cartella api preparano le richieste; il client Axios centralizza base URL, Bearer e gestione del 401.

Le API reali sono la modalità predefinita. La configurazione mock resta disponibile come dimostrazione e non riproduce tutte le autorizzazioni del backend. Le guardie frontend migliorano la navigazione: il server controlla comunque ogni operazione. Il dettaglio carica l'immagine separatamente come Blob, senza inserirla nel JSON della issue.

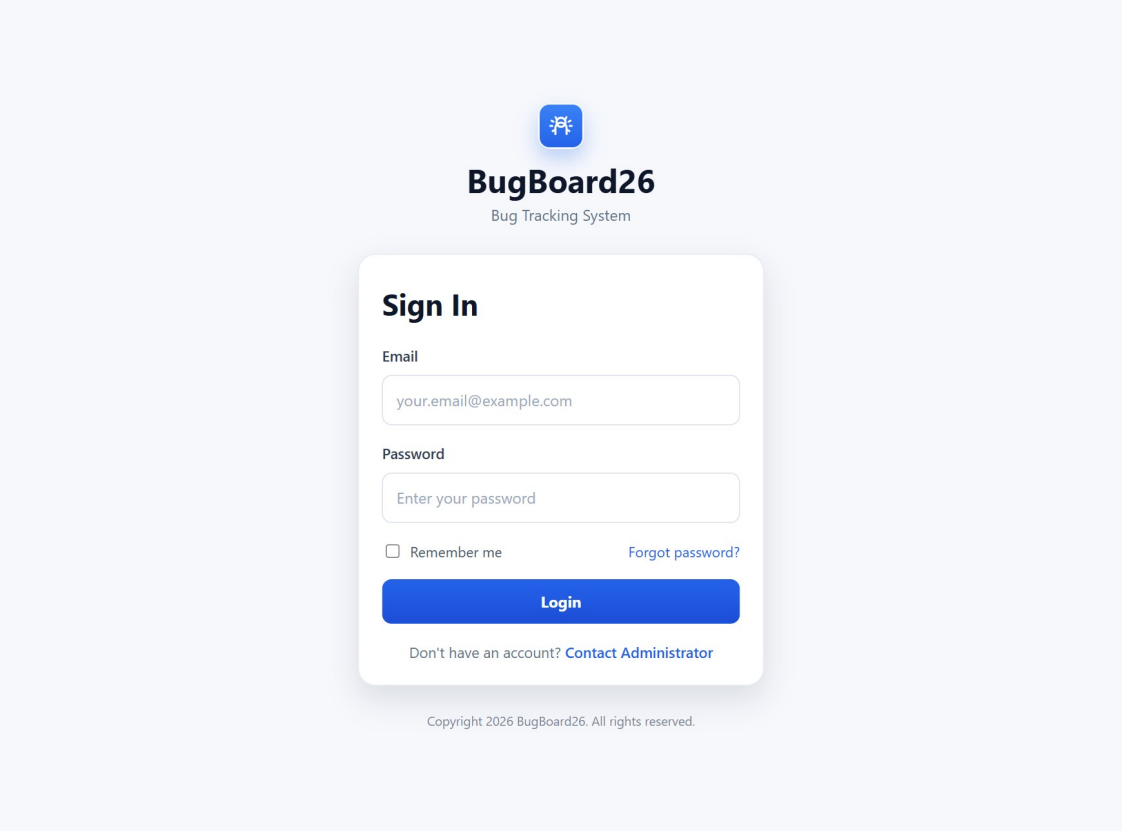


Figura 5.1 Pagina di accesso della SPA con API reali

5.3 Criteri di Design e Motivazioni

Separazione delle responsabilità

La struttura app.js/server.js distingue configurazione Express e avvio. Rotte, controller, controlli condivisi e modelli sono riconoscibili senza ricreare repository, interfacce o DTO Java.

Composizione e orientamento a oggetti

Le entità sono classi Sequelize; ApiError estende Error. Controller e servizi sono funzioni ES modules. React usa componenti funzione e hook, rappresentati nei diagrammi UML come unità di responsabilità.

Compatibilità della migrazione

Il passaggio da Spring Boot a Express ha mantenuto PostgreSQL, identificativi, hash BCrypt e formato JWT. Gli endpoint usati dal frontend sono rimasti compatibili; le modifiche funzionali successive hanno introdotto priorità facoltativa, TODO, vincolo dell'assegnatario e immagini.

Atomicità e semplicità degli allegati

La tabella issue_images conserva i byte in PostgreSQL: issue e immagine vengono salvate nella stessa transazione. Le liste leggono solo i metadati; il file viene restituito da un endpoint privato. Per le dimensioni del progetto non serve un servizio cloud aggiuntivo.

Limiti della soluzione

Liste non paginate e polling delle notifiche ogni 30 secondi privilegiano comprensibilità. Prestazioni, comportamento con molto carico e accessibilità completa richiedono misure specifiche.

5.4 Tecnologie Utilizzate

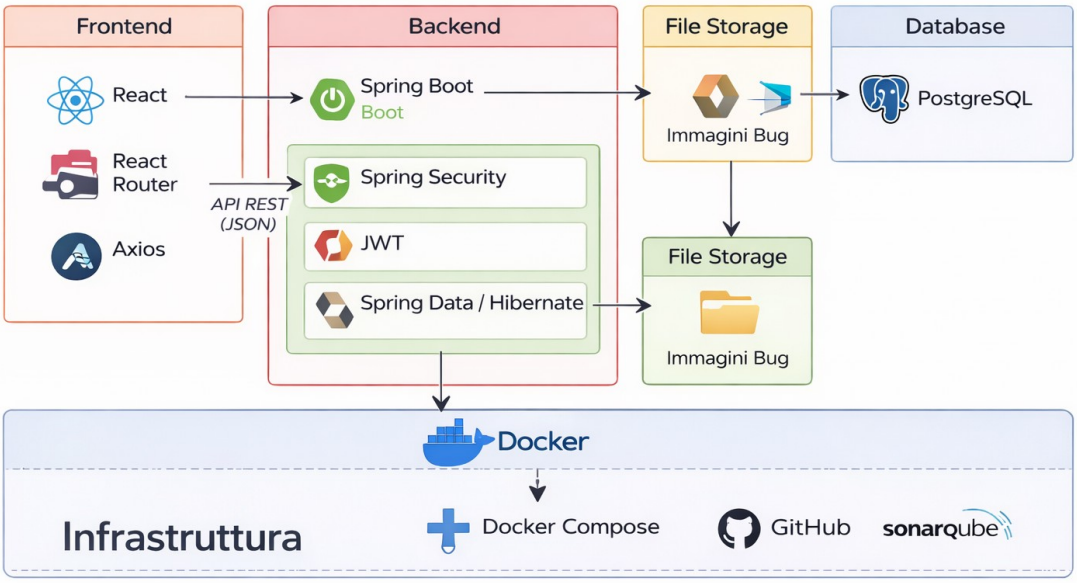


Figura 5.2 Schema tecnologico della progettazione iniziale

Area	Implementazione attuale e lettura del disegno
Frontend	React 19, React Router 7, Axios, Vite 8; la parte React del disegno resta pertinente.
Backend	Node.js almeno 22, JavaScript ESM, Express 5. Spring Boot e Spring Security nel disegno appartengono alla progettazione precedente.
Persistenza	Sequelize 6 e driver pg verso PostgreSQL 17; sostituiscono JPA/Hibernate.
Autenticazione	JWT HS256 e bcryptjs per hash BCrypt; controllo dell'account dal database a ogni richiesta.
Immagini	Multer limita il multipart; Sharp verifica e normalizza PNG/JPEG/WebP. Byte in PostgreSQL, senza cartella pubblica o storage esterno.
Infrastruttura	Dockerfile backend e Compose con backend/PostgreSQL; il frontend usa Vite locale.
Verifica	node:test per API/backend; Vitest, Testing Library e jsdom per frontend; ESLint e build. Non è configurato un servizio SonarQube.

5.5 Schema della Persistenza dei Dati

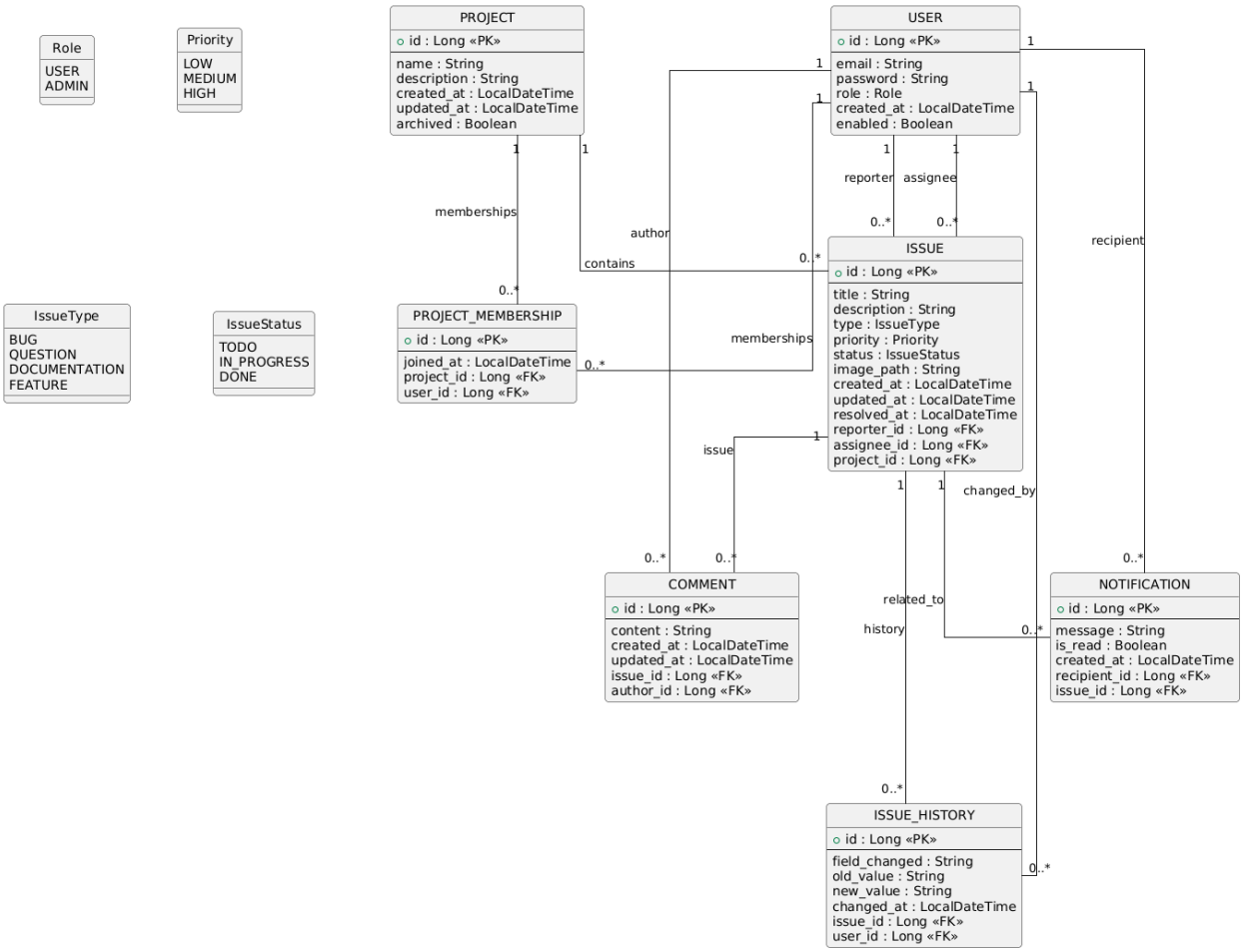


Figura 5.3 Diagramma di dominio della progettazione iniziale

Il disegno conserva il modello del progetto. Per leggere lo schema realmente usato occorre applicare le differenze della tabella seguente: non tutte le proprietà storiche sono diventate colonne e l'allegato è una relazione separata.

5.5.1 Descrizione delle entità e delle relazioni

Modello e tabella	Campi e legami principali
User / app_users	id, fullName, email unica, passwordHash, role, createdAt. Non esiste enabled; la password non è salvata in chiaro.
Project / projects	name, description, status ACTIVE/ARCHIVED, createdAt, updatedAt. Il disegno usa archived, mentre il codice ha status.
Membership / project_memberships	projectId, userId, joinedAt e coppia univoca; collega utenti e progetti molti-a-molti. Nessun ruolo per progetto.
Issue / issues	projectId e reporterId obbligatori; assigneeId facoltativo. priority nullable con anche CRITICAL; stato finale CLOSED, non DONE. Nessun resolvedAt o image_path.
IssueImage / issue_images	issueId chiave primaria e riferimento a Issue; contentType e data BYTEA. Relazione Issue 1 a IssueImage 0..1, successiva al disegno.
Comment / issue_comments	issueId, authorId, content, createdAt. Non è esposta modifica dei commenti o updatedAt.
Notification / notifications	recipientId, issueId, message, kind ACTIVITY/RESOLUTION, read, createdAt. issueId resta nel database anche se la risposta lo maschera.
ISSUE_HISTORY	Presente nel disegno storico; nessuna tabella o funzione di storico è implementata.

db:init esegue esplicitamente gli script SQL 001, 002 e 003 nella stessa transazione. 002 converte OPEN in TODO, rende la priorità nullable e introduce il tipo di notifica; 003 crea issue_images. Il server verifica il collegamento e lo schema richiesto, ma non migra automaticamente il database. Gli script ripetibili non eliminano dati esistenti.

Le notifiche di risoluzione restano personali anche dopo la revoca della membership. La risposta restituisce issueId null se il destinatario non può più aprire il dettaglio; la relazione nel database viene conservata. openIssues mantiene il nome JSON originario per compatibilità ma conta le issue TODO.

6 Design dell’interfaccia utente

6.1 Progettazione Cromatica dell’Interfaccia

La palette distingue superfici, navigazione e informazioni funzionali. L’obiettivo è aiutare la lettura di elenchi e metadati mantenendo coerenti dashboard, dettaglio e amministrazione.

6.1.1 Obiettivi della Progettazione Cromatica

I colori guidano verso le azioni principali, distinguono i tipi di issue e mantengono riconoscibile il contesto del progetto. Badge e pulsanti sono accompagnati da testo; l’utente non deve dedurre il significato soltanto dalla tinta.

6.1.2 Palette Cromatica Adottata

Uso	Valore o comportamento nel CSS
Sfondo e superfici	--page-bg #f6f8fc; --surface #ffffff; --border #d9e1ec.
Testo	--text-primary #0f172a; --text-secondary #64748b.
Azioni	--accent #2563eb; --accent-strong #1d4ed8.
Tipi e stati	Badge testuali differenziati per BUG, QUESTION, FEATURE e DOCUMENTATION. TODO, IN_PROGRESS e CLOSED hanno anche una distinzione cromatica.

6.1.3 Applicazione della Regola 60-30-10

Il modello 60–30–10 esprime una guida progettuale: prevalenza dei neutri, blu come colore identitario e accenti per le informazioni salienti. Le quote nel disegno illustrano l'intento e non sono una misura automatica dei pixel dell'interfaccia attuale.

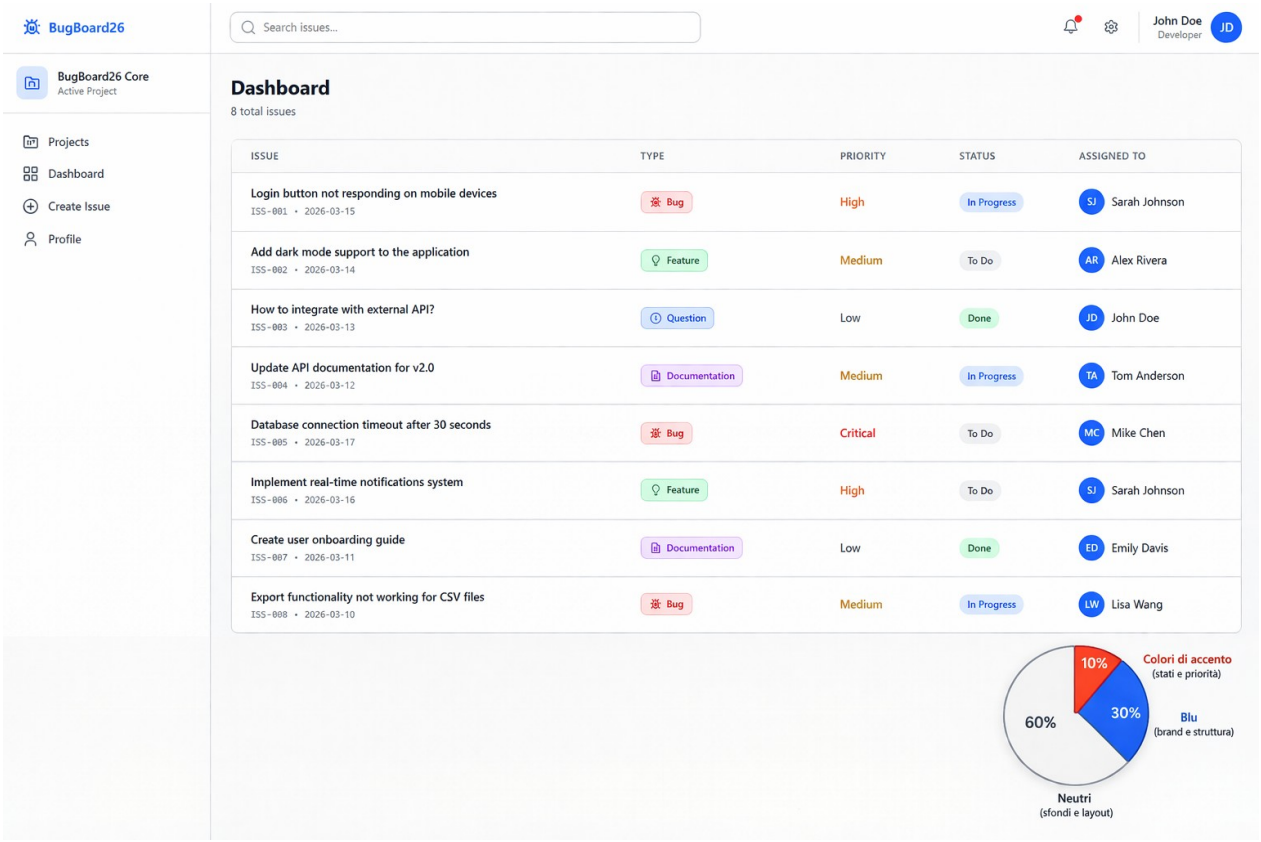


Figura 6.1 Distribuzione cromatica proposta nel progetto

6.1.4 Accessibilità e Contrasto Cromatico

Il contrasto leggibile e l'uso congiunto di testo e colore sono obiettivi del design. Una dichiarazione di conformità WCAG richiede misure sui colori renderizzati e prove dedicate: la presenza della palette e dei badge non dimostra da sola il rispetto di tutti i criteri.

6.2 Tipografia

La tipografia separa titoli, azioni e metadati e favorisce la consultazione di descrizioni e discussioni. Le stesse regole CSS sostengono le pagine principali e i componenti condivisi.

6.2.1 Obiettivi della Progettazione Tipografica

Rendere distinguibili il titolo del progetto, il titolo della issue e le informazioni secondarie; mantenere etichette vicine ai valori e consentire di leggere descrizioni e commenti senza confonderli con le azioni.

6.2.2 Scelta del Font

Il CSS dichiara Inter, Segoe UI e sans-serif. Il browser usa il primo font disponibile: nel progetto non è dimostrato il caricamento di un file Inter dedicato, quindi la resa può dipendere dall'ambiente. Il font della relazione è distinto dal font della UI.

6.2.3 Gerarchia Tipografica

Titoli di pagina e issue usano dimensione e peso maggiori; etichette, date e ruoli hanno un livello subordinato. Badge e intestazioni restano leggibili nel contesto delle rispettive card senza trasformare tutti i dati in elementi di uguale rilievo.

6.2.4 Spaziatura e Allineamento

Allineamenti condivisi e distanze regolari collegano intestazioni, campi e pulsanti. La tabella delle issue permette il confronto per colonne; il dettaglio separa contenuto e metadati attraverso un pannello laterale.

6.2.5 Accessibilità e Conformità alle WCAG

Ridimensionamento, contrasto, navigazione da tastiera e ordine del focus devono essere verificati sulle schermate reali. La leggibilità percepita nelle figure non equivale a una certificazione di accessibilità.

6.2.6 Coerenza con l'Identità Visiva

La combinazione di blu, superfici chiare e tipografia sans-serif mantiene riconoscibili logo, navigazione e comandi tra le viste. La coerenza deve essere valutata anche nei messaggi di errore e negli stati disabilitati.

6.3 Principles of Proximity and Connectedness

La prossimità raggruppa gli elementi vicini; la connectedness li associa attraverso un contenitore o un collegamento visivo. I due principi spiegano la disposizione di card, form e pannelli del progetto.

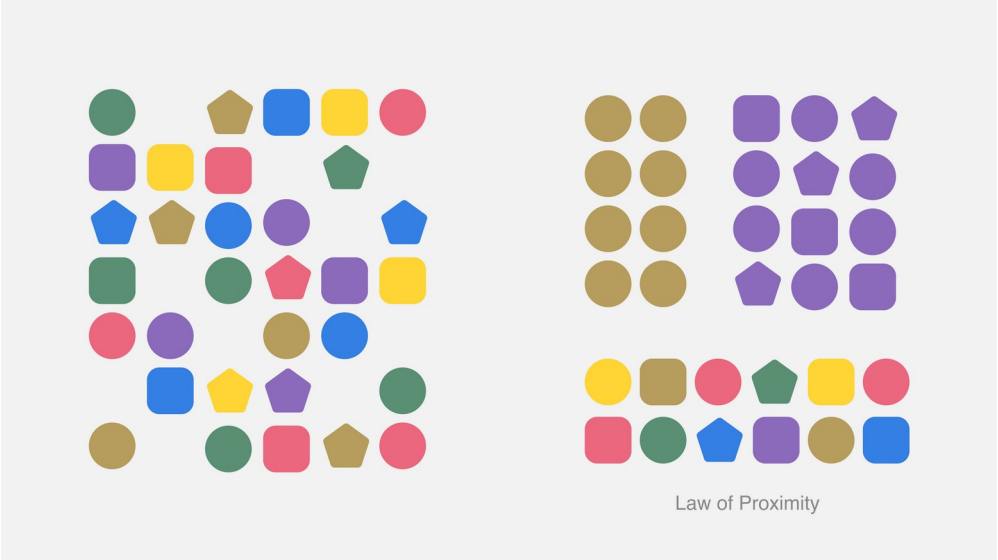


Figura 6.2 Esempio progettuale del principio di prossimità

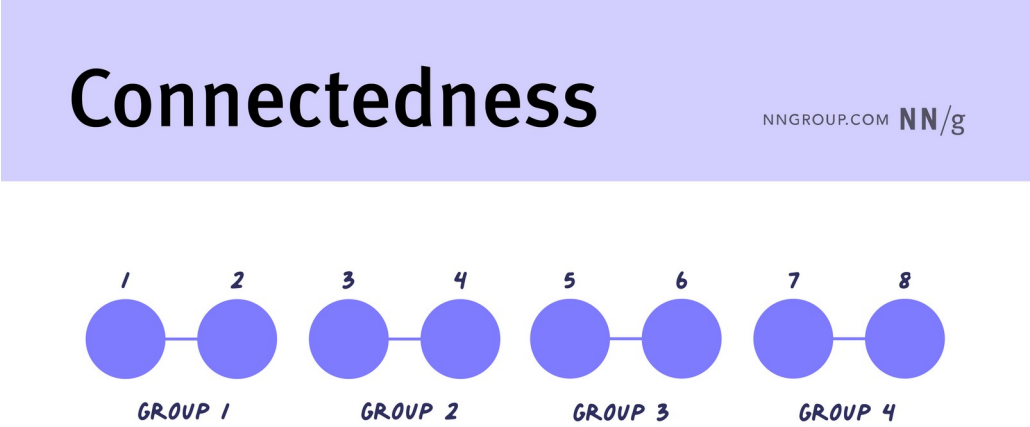


Figura 6.3 Esempio progettuale del principio di connectedness

Applicazione ai componenti di BugBoard26

Sidebar e contesto del progetto formano un insieme riconoscibile; le card dei progetti raggruppano nome, descrizione e conteggi. La lista delle issue mantiene i dati di ogni segnalazione sulla stessa riga.

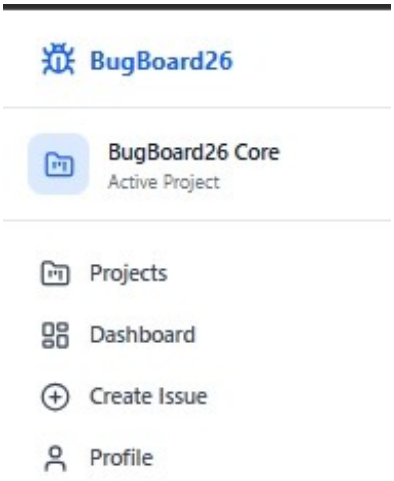


Figura 6.4 Raggruppamento della navigazione nella sidebar

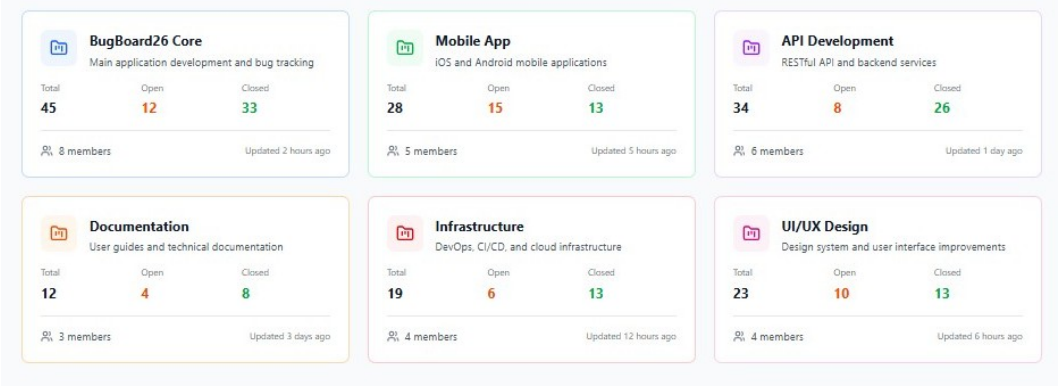


Figura 6.5 Card dei progetti nel design dell'interfaccia

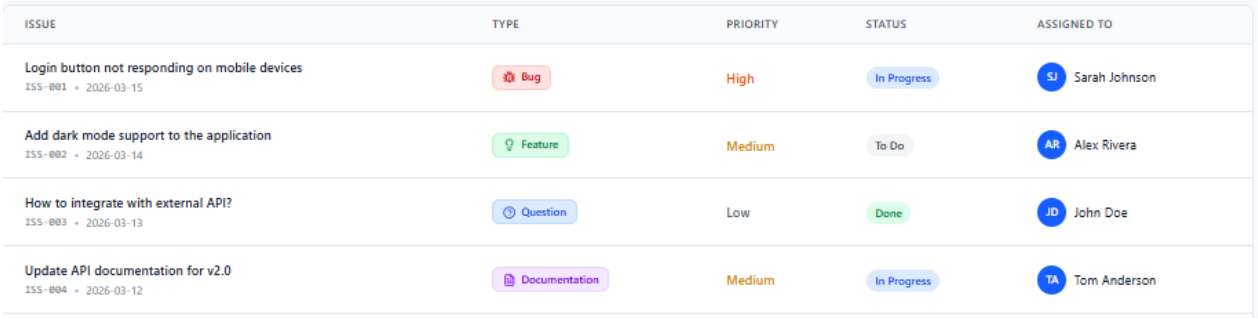


Figura 6.6 Raggruppamento dei dati nelle righe delle issue

Raggruppamento dei campi e delle azioni

Il modulo di creazione avvicina dati principali, assegnazione e immagine. La gestione utenti mantiene vicine creazione account e aggiunta di un account esistente, distinguendo i due comandi.

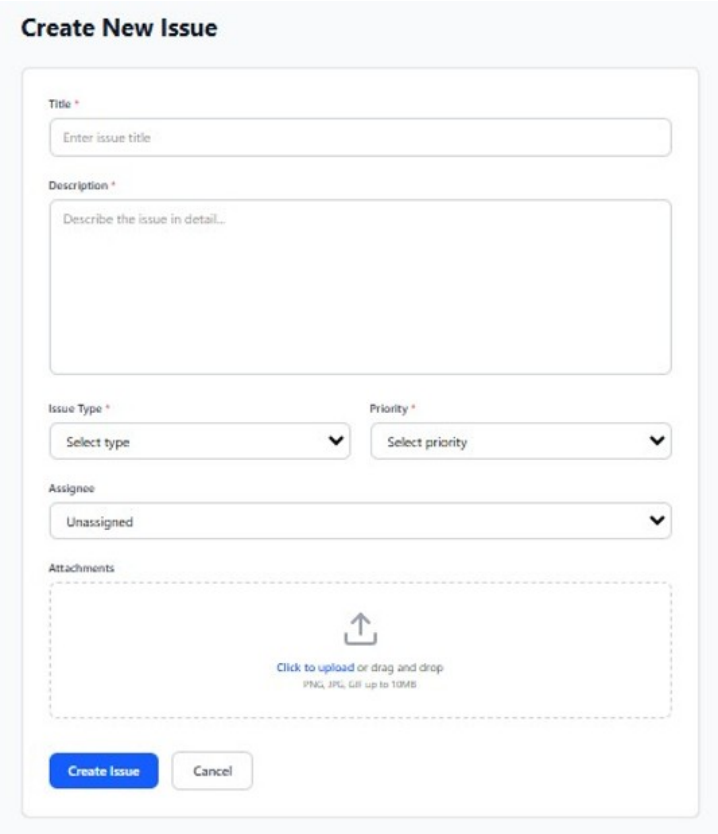


Figura 6.7 Mockup dei gruppi di campi del modulo Create Issue

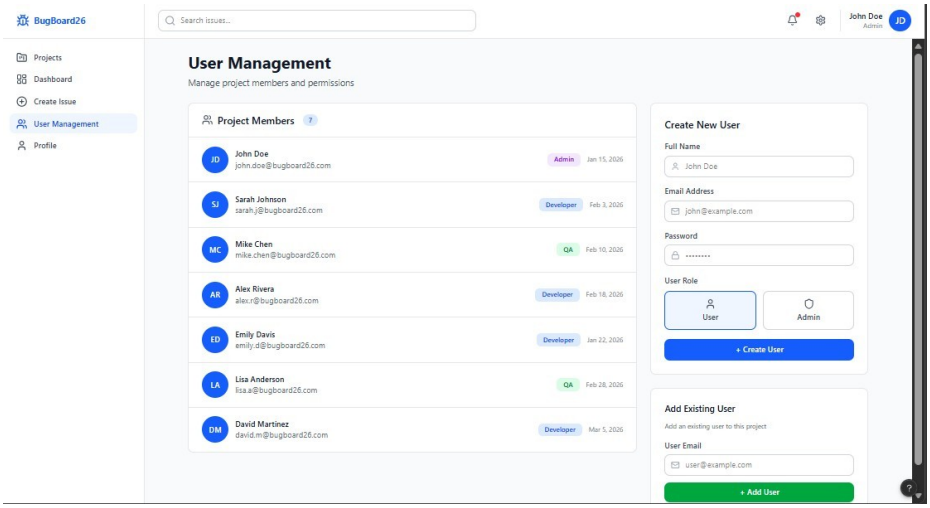


Figura 6.8 Mockup dei gruppi di azioni in User Management

La versione realizzata usa una modale per la creazione e rende facoltativa la priorità. La somiglianza dei gruppi non implica che ogni controllo dei mockup sia operativo; stato, immagini e permessi seguono i requisiti del capitolo 4.

6.4 Applicazione del modello F-Pattern

Il disegno evidenzia una scansione proposta del titolo, del contesto e delle righe delle issue. Il percorso a F costituisce una motivazione del layout; non deriva da eye-tracking o prove sperimentali sullo sguardo degli utenti.

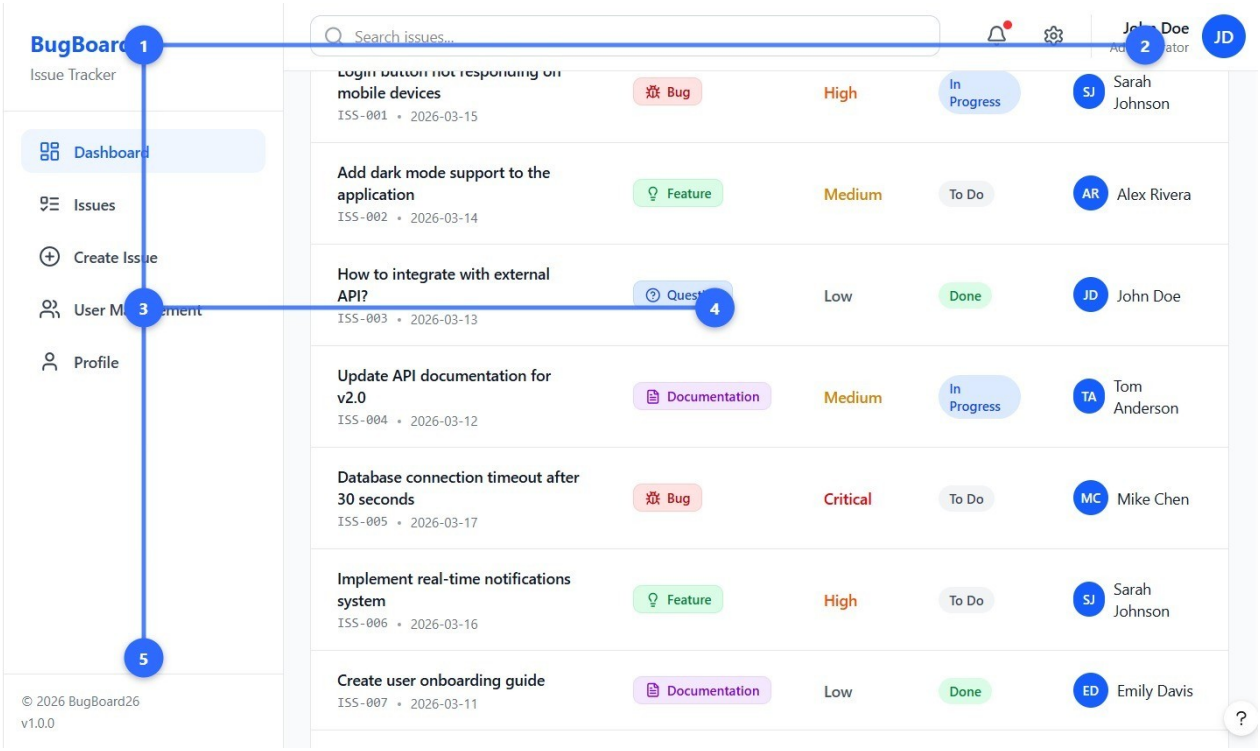


Figura 6.9 Percorso F-pattern proposto per la dashboard

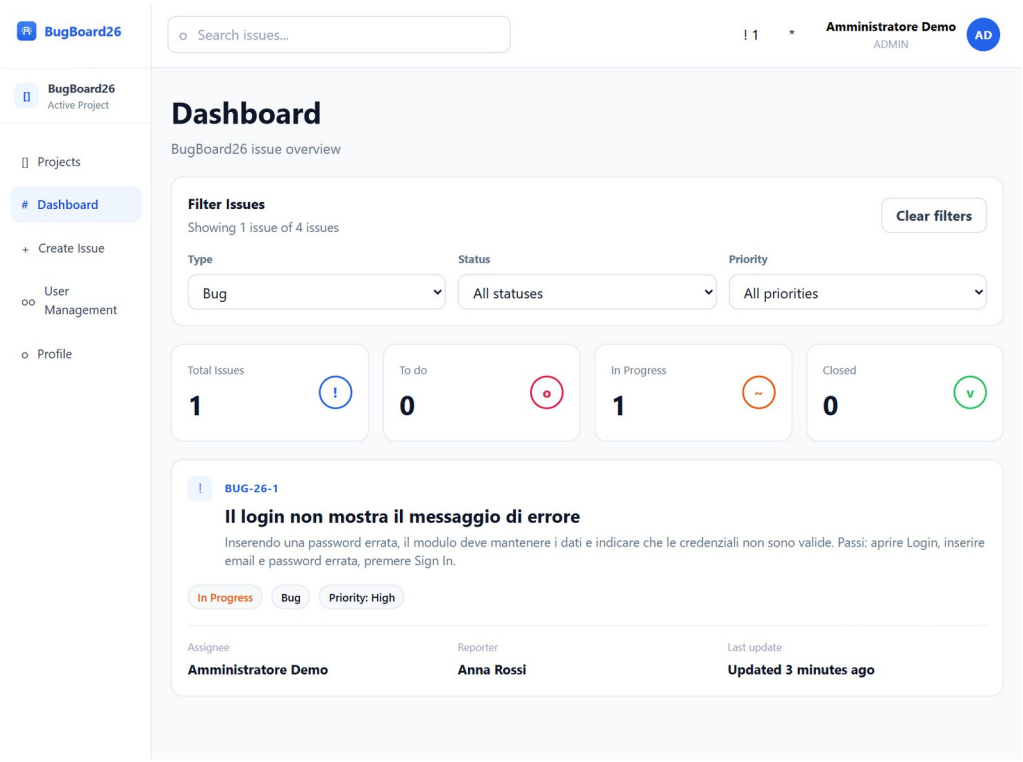


Figura 6.10 Dashboard attuale filtrata per tipo di issue

6.5 Labor Perception Bias

Il sistema rende percepibile l'elaborazione con messaggi di caricamento e pulsanti come Creating... o stati di invio disabilitati. Questi segnali aiutano a capire che la richiesta è in corso; non viene introdotta un'attesa artificiale. Il loro effetto sull'esperienza deve essere verificato con utenti.

6.6 Gestione del Feedback Utente e Feedback Persistence

I mockup di successo ed errore illustrano la proposta grafica iniziale. La creazione reale chiude la modale e apre il dettaglio, senza mostrare una finestra separata di congratulazioni. Gli errori restano nel form; un commento fallito conserva il testo e un caricamento immagine fallito offre Retry.

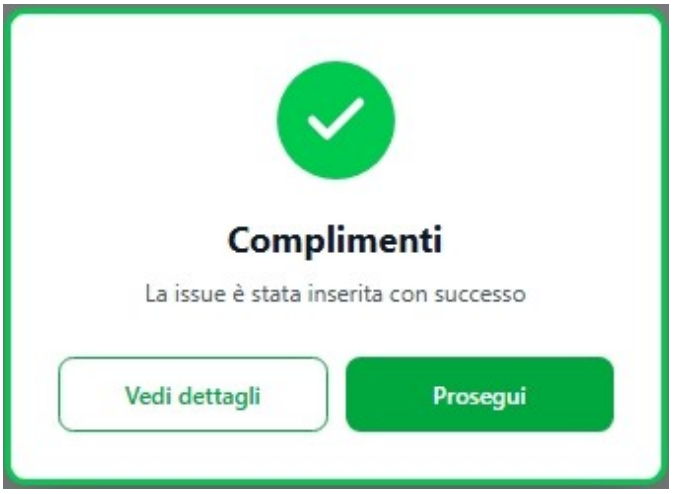


Figura 6.11 Mockup della conferma di successo



Figura 6.12 Mockup della segnalazione di errore

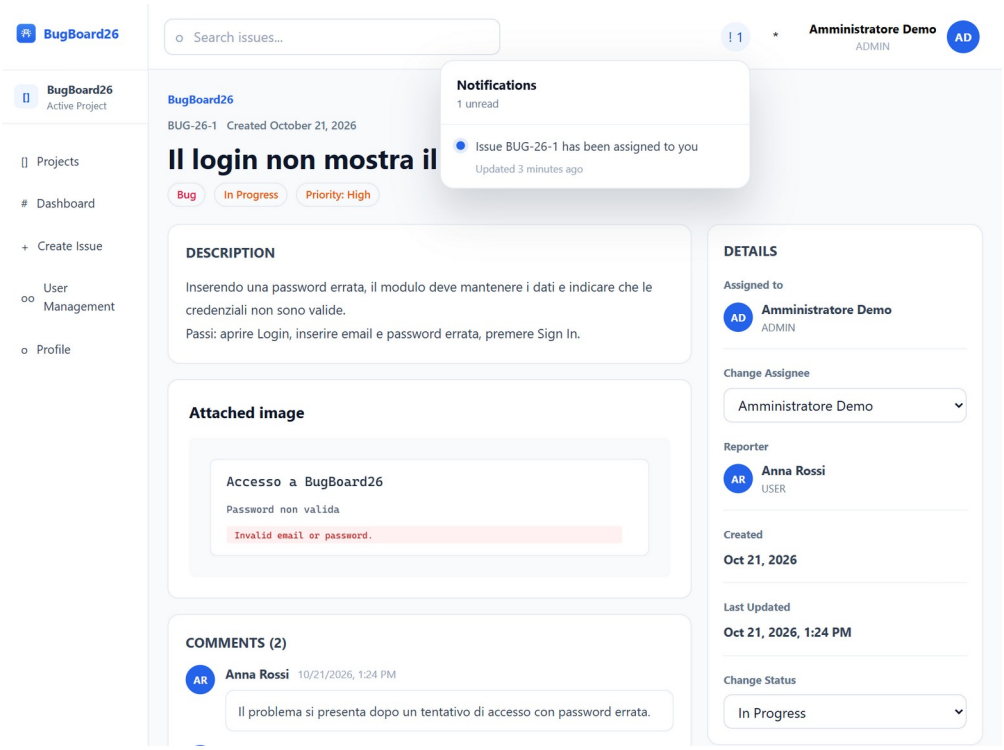


Figura 6.13 Feedback persistente tramite notifiche personali dell'app

Il contesto AuthContext aggiorna notifiche e conteggio non lette dopo le operazioni e periodicamente. Una risoluzione destinata a un ex membro resta leggibile senza collegamento alla issue non più accessibile.

6.7 Prevenire gli Errori

La UI anticipa errori con campi obbligatori, controllo del file, azioni disabilitate durante l'invio e scelta degli stati consentiti. Il backend ripete i controlli indipendentemente dal browser. Il controllo dello stato è disabilitato per i non assegnatari e per le issue CLOSED.

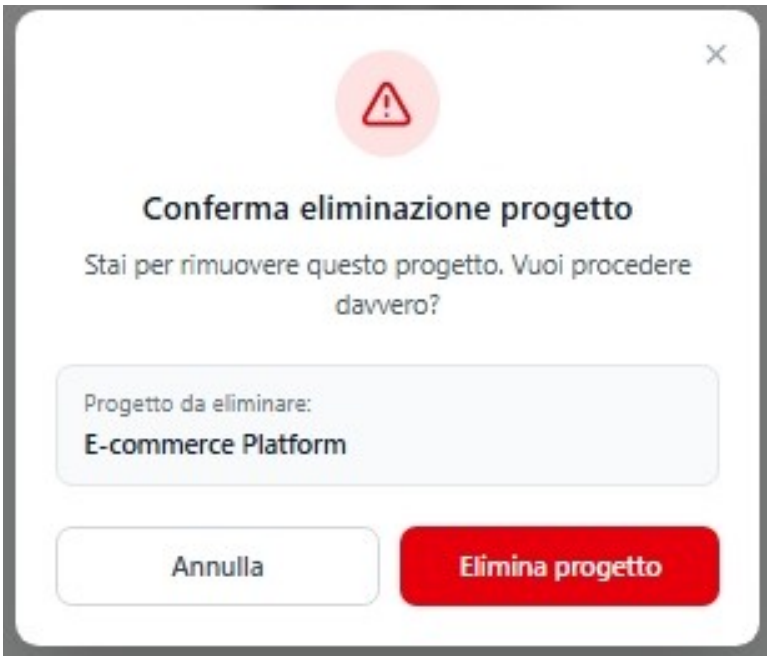


Figura 6.14 Proposta di conferma eliminazione progetto: funzione non implementata

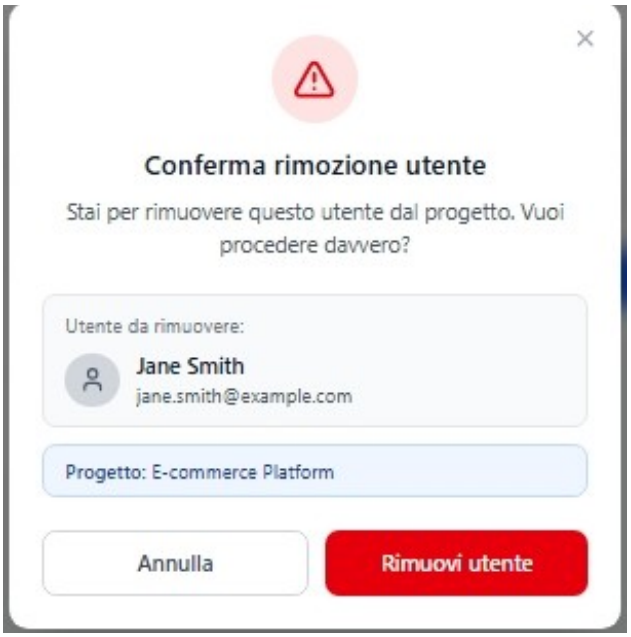


Figura 6.15 Proposta di conferma rimozione membro

La rimozione dei membri è implementata con window.confirm, diverso dalla modale illustrata. Il progetto non espone eliminazione o abbandono. Profile, Settings, ricerca in testata, Remember me e recupero password restano elementi dimostrativi o incompleti; la review dovrà valutarne l'effetto sulle aspettative.

7 Diagrammi delle classi di Design

7.1 Classi di Design del Backend

7.1.1 Architettura Controller–Service–Model

Il backend usa moduli funzionali per controller e servizi e classi Sequelize per le entità. La separazione Controller–Service–Model descrive responsabilità e dipendenze concrete, senza introdurre repository Java non presenti nei sorgenti.

Modulo	Funzioni e responsabilità
app.js, server.js, routes.js	Configurazione Express; verifica database e avvio; associazione metodo/percorso/middleware.
controllers/	auth, users, projects, members, issues, comments, notifications: input e risposta HTTP.
services/issues.js	createIssue(user,input,file), changeStatus(user,id,status), changeAssignee(user,id,assigneeId), touchProject.
services/access.js	requireProject, requireIssue con lock durante una transazione, requireAssignee, visibleProjectIds.
services/notifications.js	notify, notifyAssignment, notifyParticipants, notifyResolution.
services/images.js	prepareImage(file): controllo formato reale, normalizzazione e rimozione metadati.
services/responses.js	Funzioni di serializzazione pubblica; nessuna scrittura.
models/index.js	User, Project, Membership, Issue, IssueImage, Comment, Notification e relazioni.

7.1.2 DTO

Un DTO descrive quali dati attraversano il confine tra frontend e backend. Non è necessario implementarlo come classe: qui i controller selezionano e validano i campi di richiesta e services/responses.js costruisce gli oggetti JSON di risposta. I modelli ORM restano interni e gli hash delle password non vengono inviati al client.

Il diagramma distingue Request DTO, Domain DTO e Response DTO. I nomi conservano la proposta del progetto; i campi da leggere nella versione attuale sono precisati nelle tabelle successive. Tipi Java come Long e LocalDateTime esprimono il disegno iniziale: il contratto corrente usa valori JSON e date serializzate.

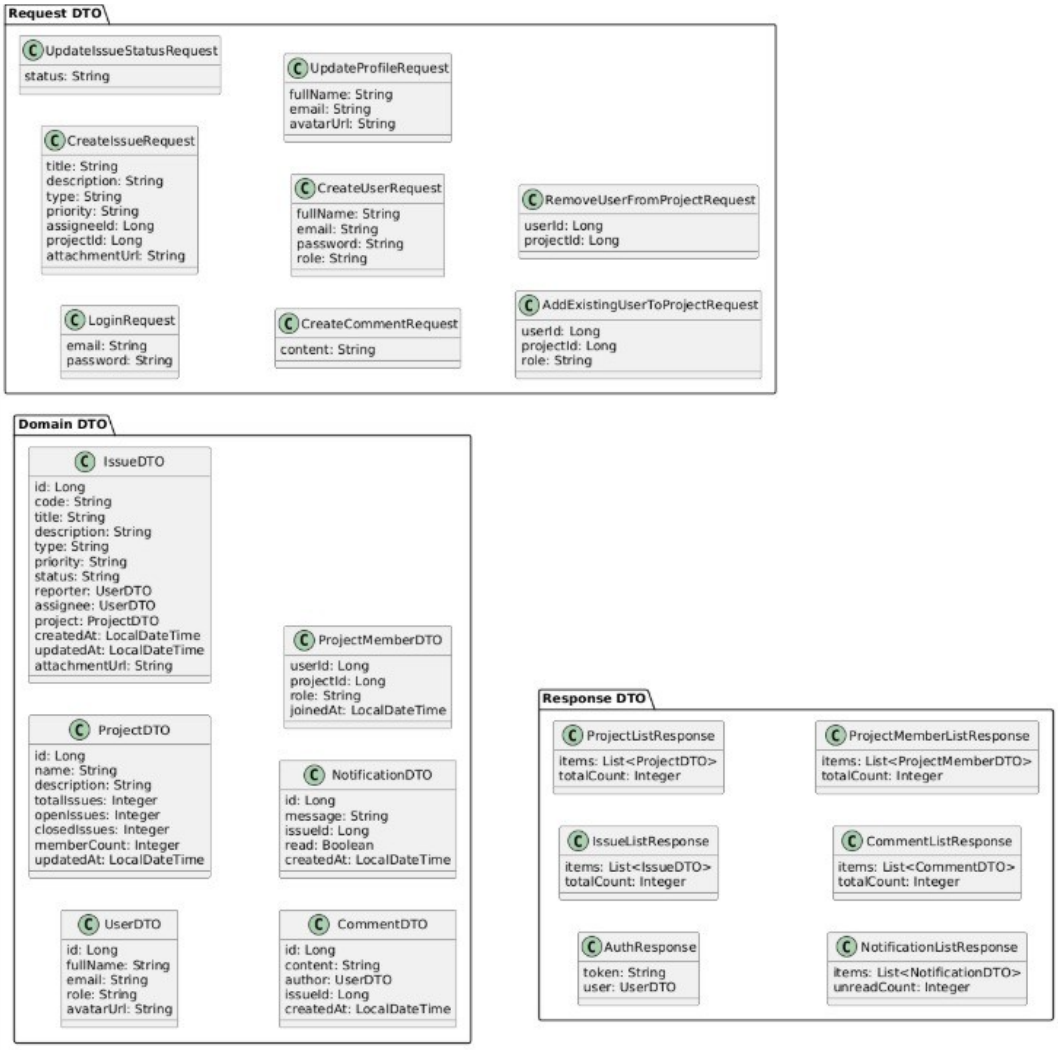


Figura 7.1 Diagramma dei DTO di richiesta, dominio e risposta

Request DTO e richieste effettive

Contratto nel diagramma	Richiesta attuale con prefisso /api
LoginRequest	POST /auth/login: email, password.
CreateUserRequest	POST /users: fullName, email, password, role. ADMIN obbligatorio.
CreateIssueRequest	POST /issues: projectId, title, description, type, priority nullable, assigneeId nullable. Con file: multipart image; attachmentUrl non è accettato come allegato.
UpdateIssueStatusRequest	PATCH /issues/{id}/status: status. Il server verifica assegnatario e transizione.
Cambio assegnatario	PATCH /issues/{id}/assignee: assigneeId oppure null; contratto successivo alla figura.
CreateCommentRequest	POST /issues/{id}/comments: content; autore e data ricavati dal server.
AddExistingUserTo ProjectRequest	POST /projects/{id}/members: userId. projectId è nel percorso; role non modifica alcun ruolo di progetto.
RemoveUserFromProjectRequest	DELETE /projects/{id}/members/{userId}: gli ID sono nel percorso, non in un DTO body.
UpdateProfileRequest	Proposta non implementata: esiste un adapter frontend, ma nessun PUT /users/me nel backend.
Creazione progetto	POST /projects: name, description; contratto reale non mostrato nella figura.

Domain DTO e serializzazione pubblica

Funzione in responses.js	Oggetto restituito
userResponse	id, fullName, email, role, avatarUrl. avatarUrl è stringa vuota; passwordHash escluso.
projectResponse	id, name, description, status, memberCount, totalIssues, openIssues, closedIssues, createdAt, updatedAt.
memberResponse	projectId, userId, fullName, email, role, joinedAt. role è il ruolo globale dell'account.
issueResponse	id, code, title, description, type, priority, status, project {id,name}, reporter, assignee nullable, image nullable, createdAt, updatedAt.
Metadati immagine	image vale null oppure {contentType}. I byte si scaricano da GET /issues/{id}/image; non esiste attachmentUrl pubblico.
commentResponse	id, issueId, content, author tramite userResponse, createdAt.
notificationResponse	id, issueId eventualmente null, message, read, createdAt. kind resta un dettaglio usato internamente.

La stessa issue può avere una rappresentazione persistente con chiavi esterne e una risposta con progetto e persone già incorporati. loadIssueResponse rilegge queste relazioni dopo una modifica; la serializzazione prepara dati e non esegue un nuovo salvataggio.

Response DTO ed errori

Risposta	Forma attuale
AuthResponse	{token, expiresAt, user}; expiresAt si aggiunge ai campi rappresentati nel diagramma.
Liste	{items, totalCount} per utenti, progetti, membri, issue e commenti.
NotificationListResponse	{items, totalCount, unreadCount}; include anche totalCount rispetto al disegno.
Errore API	{timestamp, status, code, message, path, errors}; errors contiene eventuali dettagli dei campi.
Allegato	Risposta binaria con Content-Type image/png, image/jpeg o image/webp; Cache-Control private, no-store e X-Content-Type-Options nosniff.

Gli esiti HTTP principali sono 200 per letture e modifiche, 201 per creazione e 204 per rimozione membro. Dati errati producono 400, token invalido 401, permesso negato 403, risorsa non disponibile 404, conflitto 409 e upload troppo grande 413.

```
POST /api/issues
Authorization: Bearer <token>
Content-Type: application/json

{"projectId":1,"title":"Errore nel login",
 "description":"Il messaggio non è chiaro.","type":"BUG",
 "priority":null,"assigneeId":2}
```

Nell’esempio gli ID sono dimostrativi. Il server stabilisce reporter e TODO e restituisce l’oggetto issue; non rimanda l’intera istanza Sequelize. In multipart il client omette i campi null e aggiunge image come file.

7.1.3 Entità di Dominio e sicurezza

Le entità persistenti sono le sette classi del capitolo 5.5. La configurazione Sequelize usa variabili d'ambiente e il driver pg per comunicare con PostgreSQL. La tabella separata IssueImage rende l'allegato opzionale ed evita di leggere BYTEA nelle liste.

Il login cerca l'account per email normalizzata e confronta BCrypt, mantenendo compatibilità con gli hash precedenti. createToken firma un JWT HS256 con subject, issuer, audience e scadenza. authenticate verifica il token e ricarica l'utente, così il ruolo effettivo non dipende da dati modificabili nel browser.

Il browser conserva la sessione in localStorage e invia il token nell'header Authorization; non ci sono cookie di sessione Express. Il logout rimuove i dati locali e il Bearer. Un token già emesso rimane valido fino alla scadenza se usato separatamente; il codice non mantiene una lista di revoca.

CORS limita le origini del browser ma non sostituisce l'autorizzazione. uploadImage limita una sola immagine a 5 MB; prepareImage decodifica contenuto reale entro 16 megapixel, rifiuta file non supportati o animati e rimuove i metadati. Le letture dell'allegato verificano di nuovo l'accesso alla issue.

7.2 Classi di Design del Frontend

Il diagramma raccoglie Core Frontend, Pages e Shared Components. Le caselle rappresentano responsabilità e comportamenti: nel codice React sono componenti funzione, hook e moduli, non classi che espongono necessariamente tutti i metodi UML. Le dipendenze mostrano composizione e utilizzo.

La figura completa è seguita da quattro dettagli parzialmente sovrapposti della stessa immagine originale, per leggere i componenti senza ridisegnare il diagramma. Le tabelle distinguono i nomi progettuali dai file effettivi.

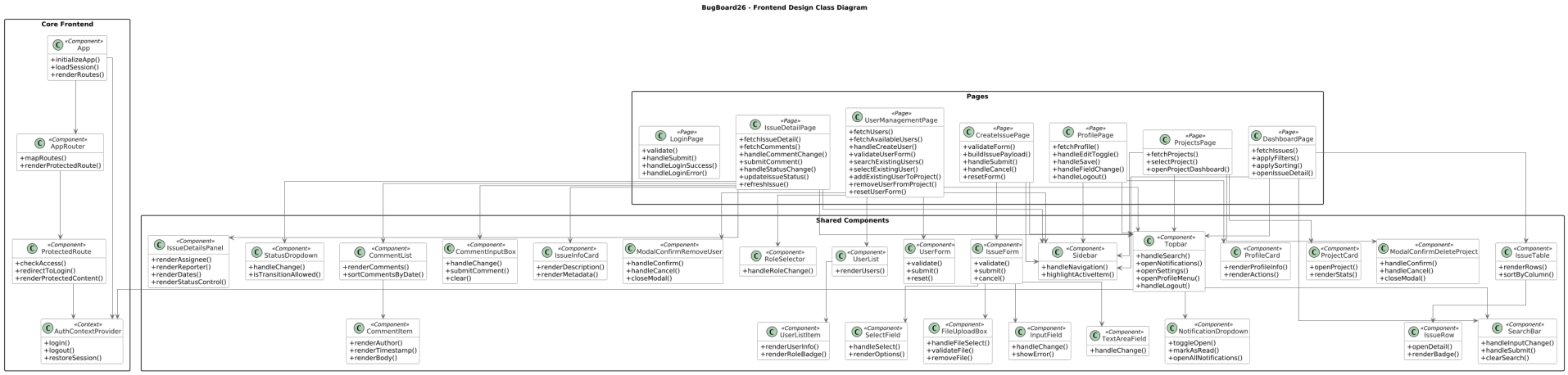


Figura 7.2 Diagramma completo delle classi di design frontend del progetto

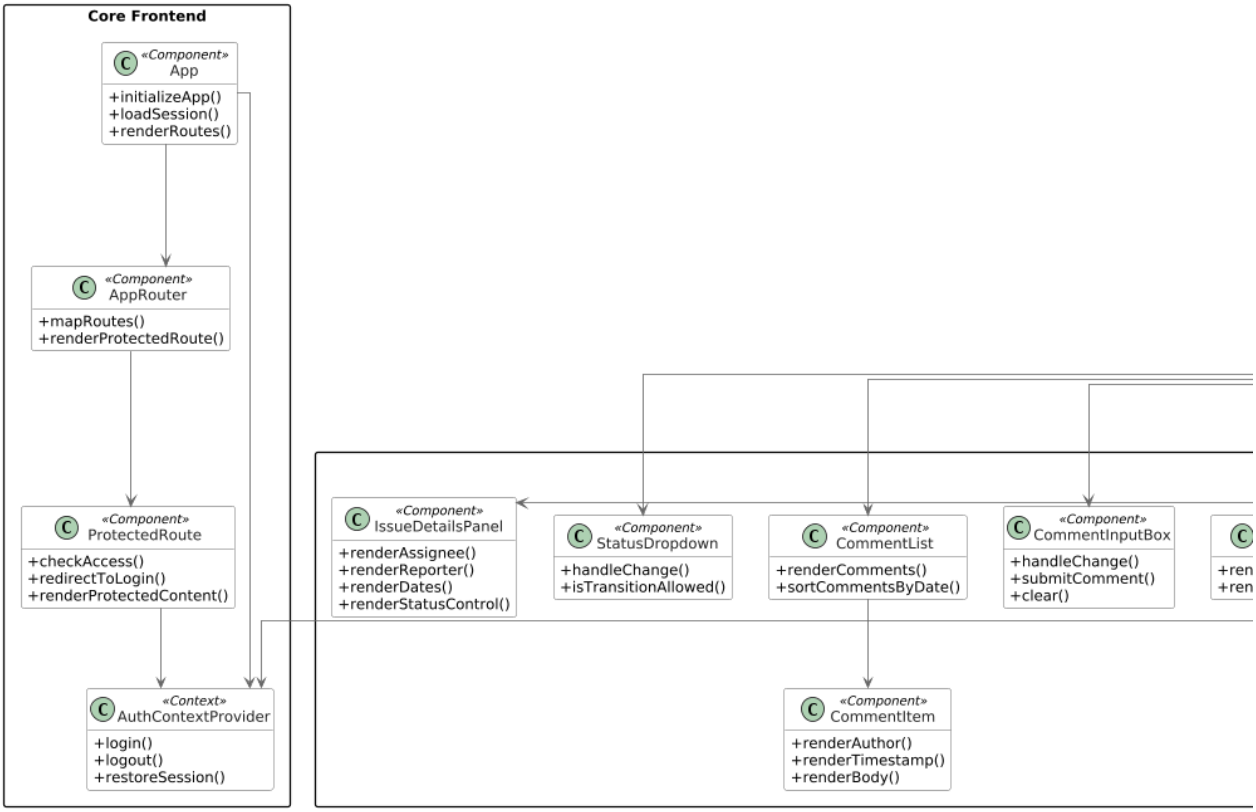


Figura 7.3 Dettaglio 1/4 del diagramma frontend originale

Core Frontend: App, router, controllo delle rotte e provider di autenticazione. Le caselle laterali appartengono allo stesso disegno completo e proseguono nei dettagli successivi.

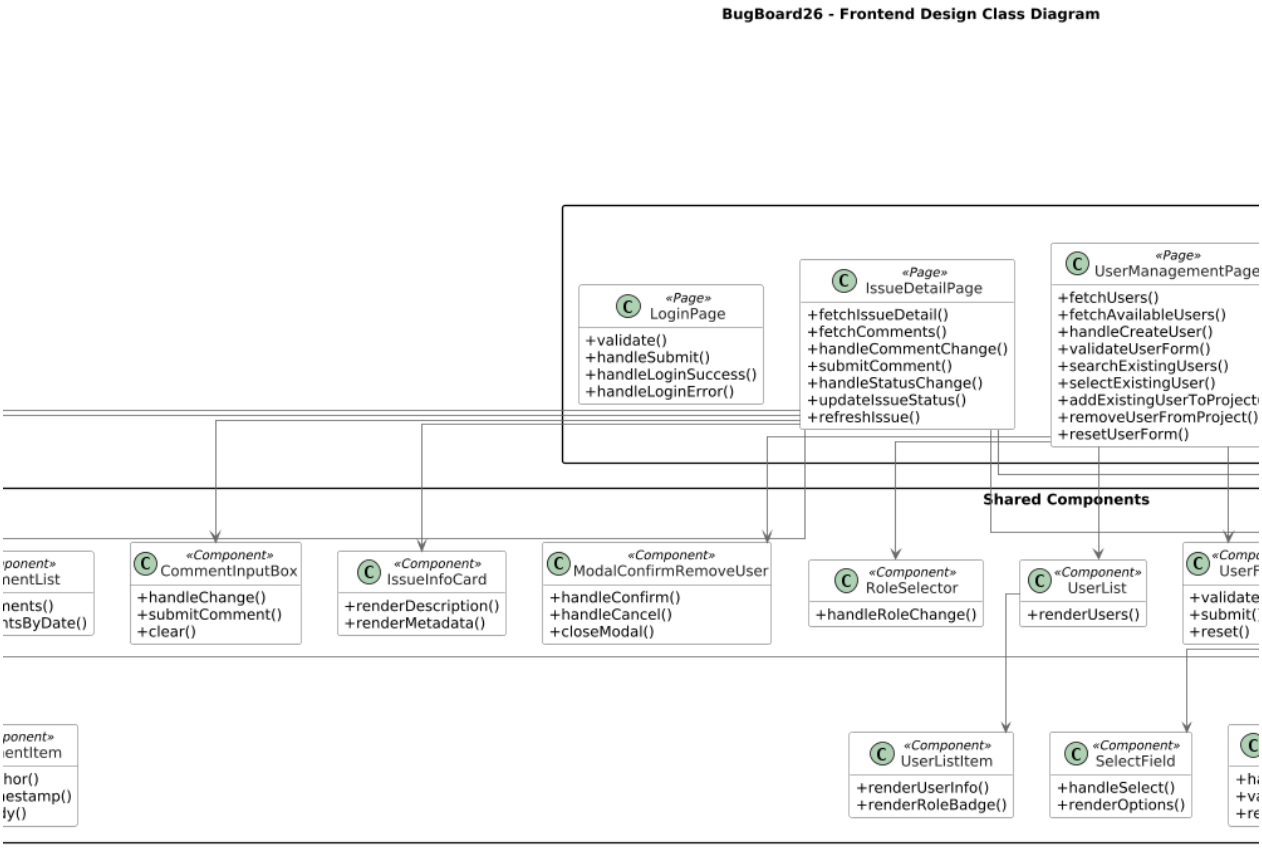


Figura 7.4 Dettaglio 2/4 del diagramma frontend originale

Pagine e componenti di dettaglio: il contenuto della issue, i commenti e i controlli sono composti nella pagina. I nomi dei moduli effettivi sono riportati nella tabella seguente.

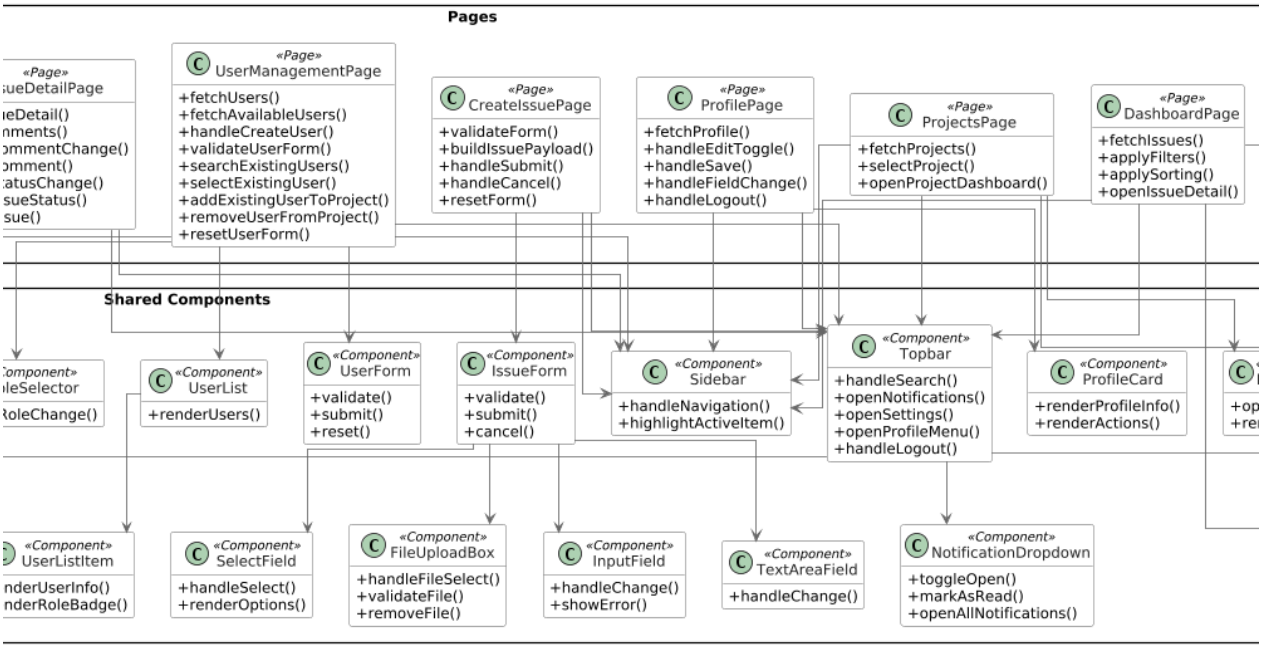


Figura 7.5 Dettaglio 3/4 del diagramma frontend originale

Pagine amministrative e form: creazione account, selezione ruolo e azioni sul progetto sono responsabilità separate. Le API conservano il ruolo globale dell'account.

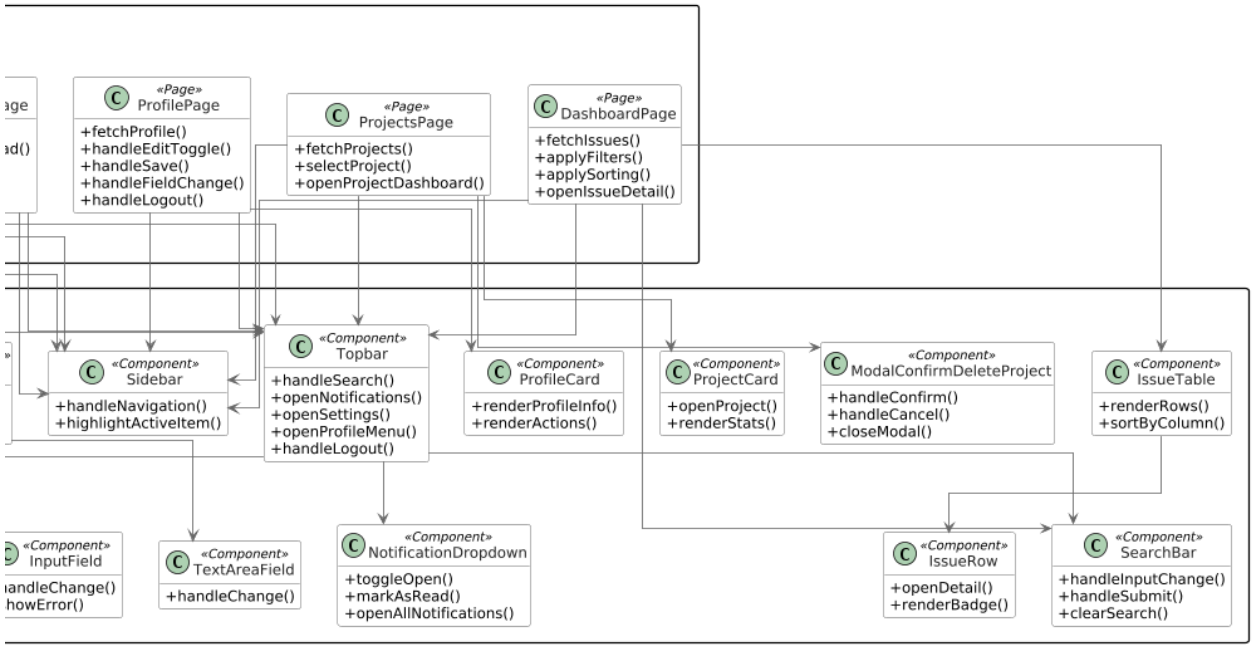


Figura 7.6 Dettaglio 4/4 del diagramma frontend originale

Navigazione e tabella: nel codice attuale alcune parti sono renderizzate direttamente dalle pagine. Le funzioni progettuali di profilo, ricerca o eliminazione progetto non sono automaticamente funzioni disponibili.

Corrispondenza tra diagramma e sorgenti

Elemento progettuale	File o comportamento effettivo
App, AppRouter, AuthContextProvider	App.jsx, router/AppRouter.jsx, context/AuthContext.jsx con AuthProvider e useAuth.
ProtectedRoute	security/RouteGuard.jsx: protezione della navigazione.
LoginPage, ProjectsPage, DashboardPage	pages/ con gli stessi nomi; lista e filtri delle issue sono in DashboardPage.
IssueDetailPage, IssueDetailsPanel	IssueDetailPage.jsx e IssueInfoCard.jsx; include stato, date, reporter e assegnatario.
CreateIssuePage, IssueForm	CreateIssueRedirectPage.jsx apre la modale CreateIssueModal.jsx; non una pagina di creazione autonoma.
CommentList e CommentInputBox	CommentsList.jsx e CommentInput.jsx; alcuni elementi di riga sono definiti all'interno del componente.
FileUploadBox	Controlli incorporati in CreateIssueModal.jsx; IssueImage.jsx gestisce il download autenticato e il rilascio dell'URL temporaneo.
UserManagementPage e componenti utenti	UserManagementPage.jsx, UserForm.jsx, UserList.jsx e RoleSelector.jsx.
Sidebar, Topbar, NotificationDropdown	Sidebar.jsx e Topbar.jsx; il pannello notifiche è interno a Topbar.
IssueTable, IssueRow, ProjectCard	Responsabilità renderizzate direttamente in DashboardPage e ProjectsPage; non file separati con questi nomi.
ProfilePage e ProfileCard	/profile apre PlaceholderPage: modifica profilo non implementata.
ModalConfirmDeleteProject, SearchBar	Eliminazione progetto e ricerca globale non implementate; i controlli del disegno non costituiscono endpoint.

7.2.1 Utility, Config e Security

I moduli trasversali supportano le pagine senza rappresentare schermate autonome. La figura mette in relazione configurazione, sicurezza, context, utility e adapter API. I simboli di legenda presenti a sinistra appartengono all’elaborato originale.

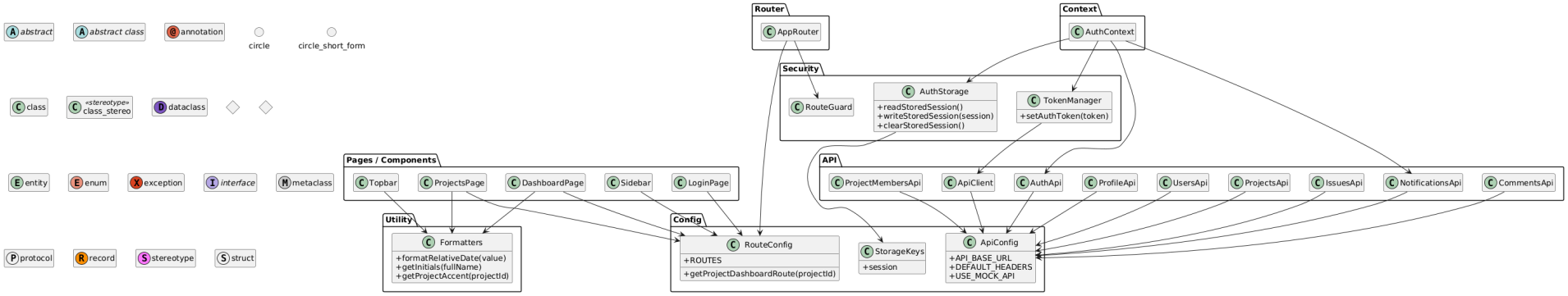


Figura 7.7 Diagramma originale Utility, Config e Security del frontend

Area	Moduli reali e compito
Utility	utils/formatters.js: date, iniziali, colori e label; include la formattazione della priorità assente.
Config	routeConfig.js, apiConfig.js, storageKeys.js: percorsi, URL base, modalità mock e chiavi locali.
Security	authStorage.js legge/scrive/cancella la sessione; tokenManager.js imposta il Bearer; RouteGuard.jsx controlla la navigazione.
Context e hook	AuthContext.jsx e useAuth.js condividono account, sessione e notifiche. Gli errori 401 azzerano la sessione.
API	client.js centralizza Axios; gli adapter auth, users, projects, projectMembers, issues, comments e notifications eseguono le chiamate.
ProfileApi	Adapter presente, ma il relativo aggiornamento profilo non dispone di endpoint backend e non è una funzione completata.

Il ripristino sessione usa GET /api/users/me; il polling notifiche è ogni 30 secondi. La modalità di accesso automatico di sviluppo è condizionata da import.meta.env.DEV e da una variabile esplicita; non costituisce un bypass del JWT nel backend.

7.3 Gestione della navigazione tra le pagine

I diagrammi distinguono percorsi USER e ADMIN. Il flusso comune parte dal login, passa ai progetti e apre dashboard e dettaglio. CreateIssuePage va letto come ingresso alla modale; ProfilePage resta un placeholder. Il percorso amministrativo aggiunge la gestione degli utenti del progetto.

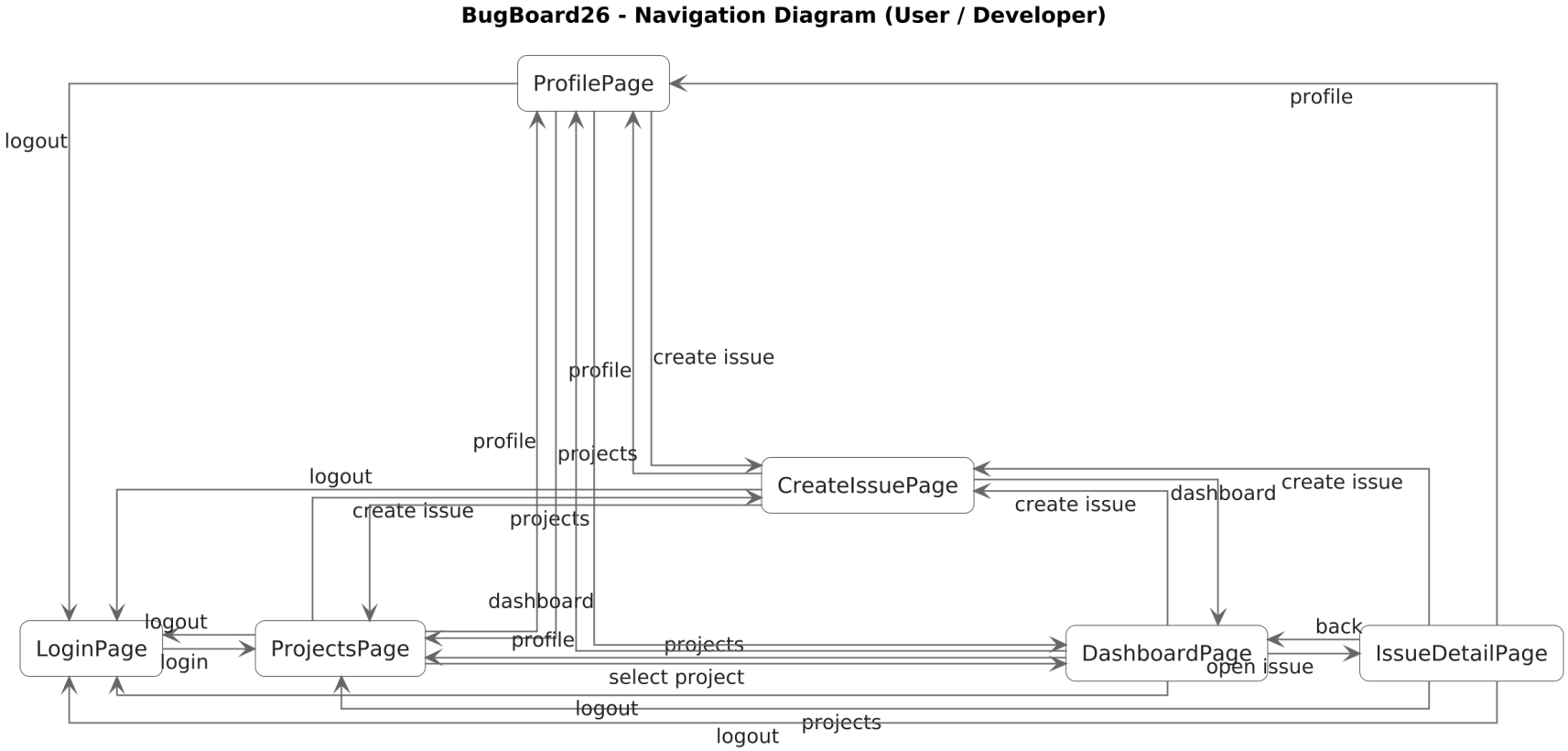


Figura 7.8 Navigazione progettuale USER / Developer

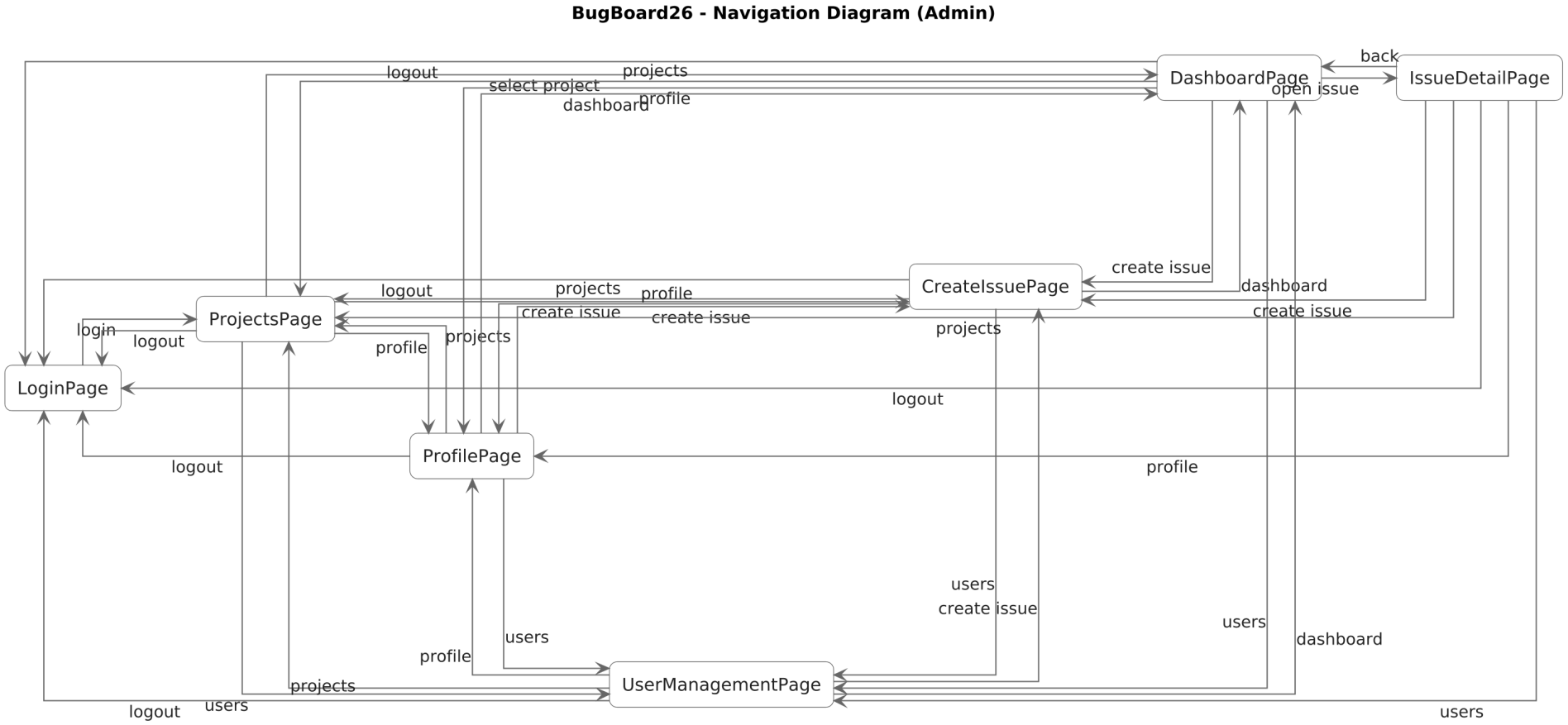


Figura 7.9 Navigazione progettuale ADMIN

Percorso React	Pagina e comportamento
/ e /login	Redirect in base alla sessione; LoginPage quando non autenticato.
/projects	ProjectsPage: progetti accessibili e creazione riservata ad ADMIN.
/dashboard/:projectId	DashboardPage: issue, filtri e modale di creazione. /dashboard senza ID richiede un contesto progetto.
/issues/:issueId	IssueDetailPage con immagine, commenti e controlli di assegnazione/stato.
/create-issue/:projectId	CreateIssueRedirectPage: redirect alla dashboard con modale aperta.
/projects/:projectId/users	UserManagementPage: la pagina verifica ADMIN; anche le API sono protette.
/profile	PlaceholderPage; nessuna modifica profilo completata.

7.4 Sequenze di Design per gli Use Case

Le sequenze seguenti descrivono i quattro flussi del capitolo 4.5 attraverso componenti e API reali. Sono espresse per passi, distinguendo preparazione della richiesta, controllo, salvataggio e risposta.

7.4.1 UC1 – Creazione issue

- CreateIssueModal raccoglie i dati; issuesApi.createIssue prepara JSON oppure FormData con il file.
- POST /api/issues attraversa authenticate e uploadImage. Il controller valida i campi e passa valori e file a services/issues.createIssue.
- Il servizio apre la transazione, verifica progetto e assegnatario, normalizza l'immagine, salva Issue ed eventuale IssueImage, aggiorna il progetto e genera la notifica di assegnazione.
- loadIssueResponse prepara il JSON pubblico. Dopo commit il controller risponde 201 e React apre il dettaglio; i byte sono scaricati successivamente tramite GET protetta.

7.4.2 UC2 – Cambio stato

- IssueInfoCard abilita il controllo soltanto per l'assegnatario. issuesApi prepara PATCH /api/issues/{id}/status.
- Il controller valida lo stato. changeStatus apre la transazione; requireIssue rilegge e blocca la riga e verifica il progetto.
- Il servizio controlla l'assegnatario prima dell'idempotenza, verifica l'avanzamento e aggiorna issue/progetto. In CLOSED crea la notifica di risoluzione al reporter.
- Un errore nella notifica annulla le scritture. Con successo la risposta 200 aggiorna il dettaglio e le notifiche senza eventi duplicati.

7.4.3 UC3 – Commento

- CommentInput invia il testo tramite commentsApi a POST /api/issues/{id}/comments.
- Il controller valida il testo, apre la transazione e verifica accesso alla issue. Comment.create esegue il salvataggio con authorId ricavato da req.user.
- Aggiorna l'attività della issue e del progetto e richiede notifyParticipants nella stessa transazione.
- La risposta 201 viene aggiunta alla discussione; il campo si svuota dopo il successo. Un errore mantiene il testo e il rollback evita salvataggi parziali.

7.4.4 UC4 – Creazione account e appartenenza

- UserManagementPage prepara POST /api/users con i dati validati dal form. La rotta richiede authenticate e requireAdmin.
- Il controller verifica email e dati, calcola l'hash e salva User. Risponde 201 tramite userResponse, senza hash.
- La pagina prepara una seconda POST /api/projects/{id}/members con userId. Il backend salva Membership e attività del progetto in transazione.
- Un fallimento dell'aggiunta non elimina l'account già creato. L'amministratore può riprendere usando Add Existing User.

Parte III Documentazione del Processo di Sviluppo

8 File di build automatica

8.1 DockerFile (Backend)

Il Dockerfile usa node:24-alpine, installa le dipendenze dal lockfile con npm ci --omit=dev e copia src, scripts e sql. Il processo è eseguito dall'utente node; espone la porta 8080 e avvia src/server.js. La build e la produzione di un PNG con Sharp nel container sono state verificate il 21 ottobre 2026.

Fase	Istruzione principale
Base	FROM node:24-alpine; WORKDIR /app
Dipendenze	COPY package.json package-lock.json ./; RUN npm ci --omit=dev
Sorgenti	COPY --chown=node:node src, scripts e sql nelle rispettive directory
Esecuzione	USER node; EXPOSE 8080; CMD node src/server.js

8.2 DockerFile (Frontend)

Non è presente un Dockerfile frontend. npm ci installa le dipendenze, npm run dev avvia Vite e npm run build produce gli asset in dist. npm run preview consente una verifica locale della build; non rappresenta un deployment di produzione. Il frontend viene avviato separatamente dal Compose.

8.3 Docker Compose

infra/docker/docker-compose.yml definisce backend e postgres. Il backend attende l'healthcheck del database; PostgreSQL usa un volume persistente. Le porte interne sono 8080 e 5432; le pubblicazioni host previste sono 8081 e 5433, da confrontare con la configurazione locale effettiva.

Servizio	Porte e dati
backend	8081 host verso 8080 container; DB_HOST postgres e DB_PORT 5432.
postgres	POSTGRES_PORT host verso 5432; postgres:17-alpine e volume postgres_data.
frontend	Nessun servizio Compose; Vite locale, normalmente porta 5173.

Prima dell'avvio controllare infra/docker/.env e conservare la configurazione già presente. Se il file manca, crearlo con POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD e POSTGRES_PORT coerenti con il database: valori dimostrativi compatibili con backend/.env.example sono riportati sotto. In infra/docker non è presente un file .env.example da copiare.

```
POSTGRES_DB=bugboard26
POSTGRES_USER=bugboard_user
POSTGRES_PASSWORD=bugboard_password
POSTGRES_PORT=5433
```

I comandi seguenti sono PowerShell dalla radice del repository. Docker Desktop deve essere disponibile:

```
docker compose -f .\infra\docker\docker-compose.yml up -d postgres
cd backend
if (-not (Test-Path .env)) { Copy-Item .env.example .env }
npm.cmd ci
npm.cmd run db:init
npm.cmd run dev
```

Nel file backend/.env impostare DB_PORT sulla porta host effettiva e le altre variabili coerenti con PostgreSQL. Con un container già presente su 5432 si usa quella porta per Node locale; tra container resta sempre 5432. Non occorre ricreare il database per cambiare stack.

In un secondo terminale, dalla radice:

```
npm.cmd --prefix frontend ci
npm.cmd --prefix frontend run dev
```

Per avviare il backend tramite immagine, dalla radice:

```
docker compose -f .\infra\docker\docker-compose.yml build backend
docker compose -f .\infra\docker\docker-compose.yml run --rm backend npm run db:init
docker compose -f .\infra\docker\docker-compose.yml up -d
```

db:init applica 001-003; il server non effettua migrazioni implicite. Non usare down -v per aggiornare dati da conservare. Le credenziali iniziali di sviluppo sono documentate nel README e vanno sostituite in un ambiente esterno.

9 Strumento di Versioning

Git conserva il codice di frontend, backend, SQL e infrastruttura nello stesso repository. Il remoto configurato è https://github.com/luxifer483/BugBoard26.git. I lockfile accompagnano le modifiche alle dipendenze, mentre file .env personali e node_modules sono esclusi dal versionamento.

Commit	Tappa dello sviluppo
8ff53a3	Autenticazione, progetti e collegamento frontend/backend.
aa15294	Gestione di issue, commenti e notifiche.
3144a8d	Migrazione a Express e Sequelize con PostgreSQL.
55fa6f9	Priorità facoltativa, stato TODO e notifiche di risoluzione.
1cf16c6	Cambio stato riservato all’assegnatario.

La versione documentata comprende anche il lavoro locale successivo su immagini e test: non va attribuito automaticamente all’ultimo commit della tabella. Prima della consegna, il commit o l’archivio definitivo dovrà identificare i sorgenti che comprendono questi file.

Prima di aggiornare il remoto si controllano git status, git diff e gli esiti delle verifiche. Le statistiche di contributo non vengono dedotte dal numero delle funzionalità: devono essere ricavate dalla cronologia effettiva, distinguendo codice, documentazione e merge.

```
BugBoard26/
  backend/       API, modelli, servizi, SQL e test HTTP
  frontend/      SPA React e test dei componenti
  infra/docker/  Compose e inizializzazione PostgreSQL
  docs/          Contratti API e diagrammi del progetto
  output/        Relazione modificabile e PDF
```

10 Qualità del codice

Le verifiche eseguite il 21 ottobre 2026 comprendono 52 test backend e 17 frontend superati, lint e build frontend riusciti, build Docker e prova di Sharp. Sono state acquisite otto schermate reali dell'app con dati dimostrativi. Non sono configurati report SonarQube, duplicazione o coverage percentuale.

Controllo	Evidenza e limite
Backend HTTP	52 test superati con PostgreSQL isolato; verifica funzionale e integrativa.
Frontend	17 test Vitest superati; componenti e richieste simulate con Testing Library/jsdom.
Lint e build	ESLint e Vite superati; verifiche sintattiche e generazione asset.
Docker	Build backend e operazione Sharp nel container verificate.
Usabilità e carico	Nessun risultato sperimentale già raccolto; protocolli del capitolo 12 da eseguire.

10.1 Grafico di Duplicazione del Codice

La misura strumentale di duplicazione non è ancora disponibile. Il grafico dovrà riferirsi al codice di consegna, indicando scanner, regole, commit e perimetro; node_modules, build e file generati devono essere esclusi. L'assenza del report non equivale a duplicazione zero.

10.2 Grafico delle Issues

Non è disponibile un report di analisi statica con conteggi di bug, vulnerabilità o code smell. Le issues di questa sezione sono segnalazioni dello strumento di qualità e non le issue applicative del database. Eventuali grafici dovranno provenire da un'analisi reale e riportare severità e criteri.

10.3 Grafico della Coverage dei Test

Il numero di test superati non è una percentuale di copertura. Non sono stati misurati linee e rami coperti: il grafico dovrà essere inserito dopo una raccolta strumentale con inclusioni, esclusioni e versione dei sorgenti esplicite. Non vengono attribuite percentuali al prodotto in assenza del report.

Parte IV Testing e Valutazione dell'Usabilità

11 Unit Testing

La verifica disponibile è composta da 52 test backend node:test e 17 frontend Vitest, superati il 21 ottobre 2026. I backend attraversano HTTP, middleware, controller, servizi e PostgreSQL reale: sono prevalentemente test di integrazione/API. I frontend verificano comportamenti dei componenti e adapter; non costituiscono una suite completa end to end.

Le sezioni 11.1-11.4 conservano le quattro funzioni selezionate e distinguono i comportamenti già verificati via API dalle specifiche di isolamento unitario. Non è implementata una suite separata di mock unitari per queste quattro funzioni. La traccia richiede un framework della famiglia xUnit: l'accettabilità formale di node:test/Vitest va confermata ai fini della consegna.

11.1 Test di createIssue (issues service)

Unità selezionata: createIssue(user,input,file) in services/issues.js. Il terzo parametro file è facoltativo ed è successivo alla proposta iniziale. Per isolarla si sostituiscono transazione, controlli accesso, modelli, prepareImage, aggiornamento progetto, notifiche e lettura risposta.

Classe o caso	Stimolo	Atteso e riscontro attuale
C1 Nominale	Membro, campi validi, assegnatario membro.	201, TODO e reporter server; verificato via HTTP.
C2 Facoltativi	Priorità, assegnatario e file assenti.	Null conservati e nessuna IssueImage; verificato API/frontend.
C3 Accesso	Utente esterno o membership revocata.	403, nessuna creazione; verificato via HTTP.
C4 Destinatario	Account non membro, anche ADMIN.	400 ASSIGNEE_NOT_MEMBER; verificato via HTTP.
C5 Immagine	PNG/JPEG/WebP; file falsi, corrotti, troppo grandi o troppi pixel.	Download valido o rifiuto senza dati parziali; otto test API dedicati.
C6 Rollback	Fallisce la notifica dopo la creazione.	Issue e immagine annullate insieme; verificato su DB.
C7 Identità	Client invia reporter, date o stato arbitrari.	Campi server protetti; verificato attraverso controller/API.

Specifica unitaria da realizzare: verificare argomenti e stessa transaction passata a tutte le dipendenze. L'atomicità reale deve restare provata su PostgreSQL; una transazione simulata non dimostra il rollback del database.

11.2 Test di changeStatus (issues service)

Unità: changeStatus(user,issueId,status). Le partizioni comprendono identità dell'attore, assegnazione, accesso e stato corrente. Il controllo dell'assegnatario precede il confronto sullo stesso stato.

Classe o caso	Stimolo	Atteso e verificato via API
S1 Assegnatario	TODO a IN_PROGRESS, poi CLOSED.	200, dati persistenti e risoluzione al reporter.
S2 Non assegnatario	Reporter o ADMIN membro ma non assegnato.	403 NOT_ISSUE_ASSIGNEE, anche per stesso stato.
S3 Non assegnata	assigneeId null.	403; l'avanzamento è possibile dopo un'assegnazione valida.
S4 Riassegnazione	Il precedente assegnatario riprova.	403; la facoltà passa al nuovo assegnatario.
S5 Transizione	TODO a CLOSED o ritorno da CLOSED.	409 INVALID_STATUS_TRANSITION; nessuna modifica.
S6 Idempotenza	Stesso stato richiesto dall'assegnatario.	200 e nessun evento duplicato.
S7 Concorrenza	Due richieste identiche di avanzamento/chiusura.	Un avanzamento e una sola notifica di risoluzione.
S8 Reporter speciale	Reporter coincide con attore o è ex membro.	Risoluzione personale sempre creata; accesso non riacquisito.
S9 Fallimento DB	Errore nella creazione della notifica.	Rollback di stato e data del progetto.

Per una futura suite unitaria isolare requireIssue, issue.update, touchProject, notifiche e loadIssueResponse. Lock e concorrenza richiedono le prove integrative già presenti; la copertura percentuale dei rami non è stata misurata.

11.3 Test di requireAssignee (access service)

Unità: requireAssignee(projectId,userId,transaction). La funzione verifica l'idoneità del destinatario dell'assegnazione; non è il controllo che autorizza l'attore a cambiare stato. Quest'ultimo si trova in changeStatus.

Classe o caso	Atteso	Evidenza
A1 userId null	Ritorna null senza cercare account o membership.	Assegnazione facoltativa verificata via API; assenza lookup da verificare unitariamente.
A2 Account assente	404 User not found.	Comportamento definito dal servizio; specifica per isolamento unitario.
A3 Utente esterno	400 ASSIGNEE_NOT_MEMBER.	Verificato via API.
A4 Membro	Restituisce l'account trovato.	Percorso nominale verificato via assegnazione/creazione.
A5 ADMIN esterno	Stesso errore del caso A3.	Verificato via API; ADMIN non esenta dalla membership del destinatario.
A6 Transaction	Entrambi i lookup ricevono la transaction.	Argomenti da verificare con stub; atomicità verificata nelle operazioni composte.

Isolamento proposto: simulare User.findByPk e Membership.findOne. Criterio white-box: coprire il ritorno anticipato, i due errori e il ritorno dell'account valido. Gli ID sintatticamente errati sono controllati a monte da validation.id.

11.4 Test di notify (notifications service)

Unità: notify(issue,actor,message,recipientIds,transaction). La funzione salva notifiche ordinarie ACTIVITY; notifyResolution applica un contratto distinto che include sempre il reporter.

Classe o caso	Atteso	Evidenza
N1 Transaction assente	Errore prima di qualsiasi scrittura.	Specifica unitaria da verificare isolando le dipendenze.
N2 Lista vuota o solo attore	Nessuna Notification.create.	Esclusione attore verificata via API; conteggio chiamate da isolare.
N3 Duplicati	Un destinatario, una notifica; valori vuoti ignorati.	Deduplicazione reporter/assegnatario verificata via API.
N4 Autorizzati	Membro o ADMIN distinto dall'attore riceve la notifica.	Regola definita dal servizio; controllo accessi coperto dalle prove API.
N5 Esclusi	Account assente o USER esterno non riceve attività.	Revoca accesso e privacy verificate via API.
N6 Errore scrittura	Errore propagato al chiamante transazionale.	Rollback reale verificato nelle operazioni sulle issue; per i commenti il codice usa la stessa transazione.

Isolamento proposto: simulare User.findByPk, Membership.findOne e Notification.create, controllando destinatari e transaction. La risoluzione del reporter attore o ex membro va verificata separatamente: è coperta da test API dedicati.

11.5 Strategie adottate

11.5.1 Controller

Le richieste HTTP della suite verificano status code, JSON, validazione, identità server e permessi. Il database è isolato in uno schema casuale. Non viene riclassificata come test unitario una prova che attraversa tutta la catena HTTP e SQL.

11.5.2 Service

Partizionamento degli input e casi limite coprono transizioni, optional, autorizzazione e fallimenti. Le specifiche di unit test aggiungeranno il controllo delle dipendenze isolate. Le prove attuali verificano invece anche gli effetti persistenti e il rollback reale.

11.5.3 Model / persistenza

I test su PostgreSQL verificano vincoli univoci, appartenenze, transazioni, migrazioni ripetibili e riavvio del processo. Gli allegati mantengono gli stessi byte dopo ripetizione di db:init e riavvio. Il database di sviluppo non viene ricreato automaticamente.

11.5.4 Security e Utils

Sono provati JWT mancanti, alterati e scaduti, ruoli riletti, account sicuri, password multibyte, ID non validi e accessi negati. Il frontend aggiunge 17 test: CreateIssueModal 7, IssueImage 3, IssueInfoCard 4 e issuesApi 3. Verificano priorità facoltativa, file, errori, download e rilascio degli URL temporanei, richieste multipart e abilitazione dello stato.

Comandi di ripetizione dei controlli

```
powershell -ExecutionPolicy Bypass -File .\backend\test-docker.ps1
npm.cmd --prefix frontend test
npm.cmd --prefix frontend run lint
npm.cmd --prefix frontend run build
```

test-docker.ps1 prepara un PostgreSQL temporaneo e rimuove soltanto il container creato. In alternativa impostare TEST_DATABASE_URL su un database dedicato ai test ed eseguire npm.cmd test nella cartella backend. Non usare un database di produzione.

11.6 Test plan di una funzionalità: risoluzione issue

Elemento	Piano
Obiettivo	Verificare chiusura, identità dell'assegnatario, notifica al reporter e mantenimento dei permessi.
Ambiente	Backend corrente, PostgreSQL isolato e frontend con API reali; registrare commit, data e versioni.
Dati	ADMIN A, reporter R, assegnatario S ed esterno X; progetto con issue IN_PROGRESS. Account distinti salvo varianti.
Precondizioni	SQL 001-003 applicati, token validi, membership e notifiche iniziali note.
Ingresso e uscita	Avvio e dati noti; tutti i casi registrati con esito reale e anomalie. Gli aspetti API sono già coperti dalla suite; il percorso completo browser va annotato separatamente.

Caso	Azione	Esito atteso
TP1	S chiude la issue.	CLOSED persistente e RESOLUTION a R.
TP2	R o A non assegnati tentano anche lo stato corrente.	403 e nessuna modifica.
TP3	Ripetizione e due chiusure concorrenti da S.	Una sola risoluzione per l'avanzamento riuscito.
TP4	Reporter attore oppure ex membro.	Notifica personale leggibile; nessun accesso riacquisito.
TP5	Esterno; salto TODO a CLOSED; issue non assegnata.	403 o 409 secondo il caso, senza salvataggi.
TP6	Errore notifica e successivo riavvio.	Rollback sull'errore; dati delle operazioni riuscite conservati.

Per la prova browser registrare per ciascun caso data, account, dati iniziali, azione, esito osservato e schermata. I risultati automatici già disponibili non costituiscono osservazioni di usabilità con utenti.

12 Valutazione dell’Usabilità

La valutazione con persone e i questionari devono ancora essere svolti. Le sezioni seguenti contengono protocollo e schede da compilare: non sono già disponibili partecipanti, punteggi, tempi o risultati sperimentali.

12.1 Expert Review

12.1.1 Obiettivo del Test

Esaminare comprensibilità e coerenza dei flussi secondo le otto regole d’oro di Shneiderman. La review considera segnalazione con immagine, filtri, commenti, assegnazione, avanzamento, notifiche e amministrazione.

12.1.2 Istruzioni per l’Uso della Checklist

Ogni valutatore esegue gli stessi compiti con dati dimostrativi, annota evidenza e severità e assegna un punteggio 1-5 o Non valutabile con motivo. Una scheda deve collegare valutazione, schermata e raccomandazione. Aggregare soltanto risposte valide e indicarne il numero.

Punteggio	Significato
1-2	Problema rilevante; descrivere impedimento e possibile correzione.
3	Supporto parziale o aspetto da migliorare.
4-5	Supporto adeguato secondo l’osservazione del valutatore.
Non valutabile	Funzione o dato non disponibile; escluso dal calcolo con motivo esplicito.

12.1.3 Risultati

Da completare dopo la review. Registrare valutatori, date, ambiente, schede compilate e criticità. Non sono ancora calcolabili medie o un esito complessivo.

Checklist dell'Expert Review

Regola	Cosa osservare	Esito / note
Coerenza	Nomi, posizioni, stati e priorità non specificata coerenti tra le viste.	Da compilare
Scorciatoie	Accesso rapido a progetti, issue e azioni frequenti.	Da compilare
Feedback informativo	Attesa, successo, errori di form/upload e download immagine riconoscibili.	Da compilare
Chiusura del dialogo	Creazione conclusa nel dettaglio, commento visibile e notifica aggiornata.	Da compilare
Prevenzione errori	Campi e file controllati, stati consentiti e conferma rimozione membro.	Da compilare
Annullamento e recupero	Cancel e Remove image; testo conservato dopo errore; limite della chiusura irreversibile chiaro.	Da compilare
Controllo utente	Ruoli, assegnazione e controlli disabilitati comprensibili; placeholder riconoscibili.	Da compilare
Memoria a breve termine	Contesto progetto, filtri e dati di reporter/assegnatario sempre identificabili.	Da compilare

Aggiungere prove dedicate di tastiera, focus, zoom e contrasto; non dedurre la conformità di accessibilità dalla media della checklist.

12.2 Relazione Sperimentale

12.2.1 Descrizione dei Partecipanti

Campione da definire. Coinvolgere utenti reali o potenziali con diversa familiarità con strumenti di issue tracking e almeno una persona che svolga compiti amministrativi. Registrare codici pseudonimi, esperienza, ruolo della prova e familiarità con BugBoard26. Numero, nomi o età non vengono attribuiti a sessioni non svolte.

12.2.2 Procedura Sperimentale

- Presentare scopo e modalità di osservazione; usare un ambiente di prova con dati fittizi.
- Assegnare account e dati iniziali comparabili, distinguendo reporter, assegnatario e amministratore.
- Leggere la consegna senza indicare i pulsanti; annotare esitazioni, errori e aiuto richiesto.
- Registrare esito e tempo; raccogliere il questionario post-task e commenti aperti.
- Ripristinare i dati tra sessioni e analizzare i problemi per compito e profilo.

Task	Consegna all’utente
U1	Apri il progetto e segnala un BUG senza priorità allegando una schermata.
U2	Trova una issue con i filtri e aggiungi un commento.
U3	Assegna la issue al membro indicato; accedi come assegnatario e porta lo stato fino alla chiusura.
U4	Accedi come reporter e verifica la notifica di risoluzione e il dettaglio dell’immagine.
A1	Crea un account, aggiungilo al progetto e rimuovilo con conferma.

12.2.3 Dati Raccolti

Non sono ancora presenti osservazioni sperimentali. La scheda deve contenere una riga per ogni task effettivamente tentato da un partecipante.

Dato	Definizione operativa
Partecipante e ruolo	Codice pseudonimo, esperienza e account usato.
Task ed esito	Completato senza aiuto, con aiuto oppure non completato.
Tempo	Secondi dalla consegna al criterio di completamento.
Errori e aiuto	Azioni errate, blocchi, recuperi e interventi del moderatore.
Soddisfazione	Punteggio 1-5 raccolto dopo il task.
Note	Commenti, aspettative ed esitazioni collegati a passo o schermata.

Partecipante	Task	Esito	Tempo	Errori / aiuto	Note
_____	_____	_____	_____	_____	_____
_____	_____	_____	_____	_____	_____
_____	_____	_____	_____	_____	_____

12.2.4 Valutazione post-task

Questionario proposto: scala da 1 (per nulla d’accordo) a 5 (del tutto d’accordo). È una scheda del progetto, non una scala standardizzata o validata.

Domanda	1-5
È stato facile completare il compito.	_____
I messaggi di attesa, successo o errore erano chiari.	_____
Ho capito l’esito dell’azione.	_____
Mi sono sentito in controllo delle operazioni.	_____
Era chiaro chi poteva cambiare lo stato della issue.	_____

Quale passaggio è stato meno chiaro? _____

Che cosa miglioreresti? _____

12.2.5 Conclusioni

Da completare dopo expert review e sessioni con utenti. Il tasso di completamento sarà calcolato sui task effettivamente tentati, separando successi autonomi e assistiti. Tempi e questionari dovranno riportare numerosità e condizioni della prova; nessuna conclusione di usabilità è già dimostrata.

Le criticità raccolte dovranno essere collegate a interventi concreti e, quando possibile, a una nuova verifica. Le immagini dei mockup documentano il design, mentre le schermate dell'app documentano la realizzazione: nessuna delle due sostituisce osservazioni con persone.

Ambito da completare	Evidenza richiesta
Autori	Inserire i nomi associati al gruppo e alle matricole già riportati.
Versione di consegna	Commit o archivio che comprenda immagini, test e documentazione aggiornata.
Qualità	Misure reali per duplicazione, analisi statica e coverage se richieste dagli artefatti di consegna.
Unit testing	Confermare il framework ammesso e completare gli eventuali test isolati previsti per le quattro funzioni.
Usabilità	Schede expert review, partecipanti effettivi, osservazioni, questionari e analisi.

Riferimenti tecnici e artefatti

Sorgenti BugBoard26: backend/src, backend/sql, backend/test, frontend/src e frontend/test; configurazione Docker e README del repository.

Elaborati grafici del gruppo: BugBoard26_Overleaf.pdf, 21 ottobre 2026. Diagrammi, personas e mockup sono ripresi da questo elaborato; le schermate reali sono acquisite dalla versione corrente con dati dimostrativi.

React: https://react.dev/learn. Express: https://expressjs.com/en/guide/routing/.

Sequelize 6: https://sequelize.org/docs/v6/. PostgreSQL 17: https://www.postgresql.org/docs/17/.

Multer: https://expressjs.com/en/resources/middleware/multer/. Sharp: https://sharp.pixelplumbing.com/.